

ARIZONA STATE UNIVERSITY
CSE 515 SLN 95433: **Multimedia and Web Databases** Fall 2023
Course Project: Phase #2 Report

Group Details: Group #15

Caleb Panikulam - cpanikul@asu.edu

Gokul Vasudeva - gvasude2@asu.edu

Joshua Martin Noronha - jnoronha@asu.edu

Kiran Sthanusubramonian - ksthanus@asu.edu

Sandipan De - sandipan@asu.edu

Shankar Harinarayanan - sharina1@asu.edu

Contents

1	Abstract	4
2	Keywords	4
3	Structure of Report	4
4	Relevant Formulas and Corresponding Implementations	5
5	The Tasks	11
5.1	Task 0	11
5.1.1	Task 0a - Goal Description:	11
5.1.2	Task 0a - Assumptions Made:	12
5.1.3	Task 0a - Implementation Approach:	12
5.1.4	Task 0a - Results:	13
5.1.5	Task 0b - Goal Description:	13
5.1.6	Task 0b - Assumptions Made:	13
5.1.7	Task 0b - Implementation Approach:	14
5.1.8	Task 0b - Interface Specifications:	14
5.1.9	Task 0b - Results:	15
5.2	Task 1	16
5.2.1	Goal Description:	16
5.2.2	Assumptions Made:	16
5.2.3	Implementation Approach:	16
5.2.4	Interface Specifications:	16
5.2.5	Results:	16
5.3	Task 2	21
5.3.1	Task 2a - Goal Description:	21
5.3.2	Task 2a - Assumptions Made:	21
5.3.3	Task 2a - Implementation Approach:	21
5.3.4	Task 2a - Interface Specifications:	21
5.3.5	Task 2a - Results:	22
5.3.6	Task 2b - Goal Description:	23
5.3.7	Task 2b - Assumptions Made:	23
5.3.8	Task 2b - Implementation Approach:	23
5.3.9	Task 2b - Interface Specifications:	23
5.3.10	Task 2b - Results:	23

5.4	Task 3 - Latent Semantics 1 (LS1)	26
5.4.1	Goal Description:	26
5.4.2	Assumptions:	26
5.4.3	Implementation Approach:	26
5.4.4	Interface Specifications:	28
5.4.5	Results:	28
5.5	Task 4 - Latent Semantics 2 (LS2)	29
5.5.1	Goal Description:	29
5.5.2	Assumptions:	29
5.5.3	Implementation Approach:	29
5.5.4	Interface Specifications:	30
5.5.5	Results:	30
5.6	Task 5 - Latent Semantics (LS3)	32
5.6.1	Goal Description:	32
5.6.2	Assumptions:	32
5.6.3	Implementation Approach:	32
5.6.4	Interface Specifications:	33
5.6.5	Results:	33
5.7	Task 6 - Latent Semantics (LS4)	34
5.7.1	Goal Description:	34
5.7.2	Assumptions:	34
5.7.3	Implementation Approach:	34
5.7.4	Interface Specifications:	35
5.7.5	Results:	35
5.8	Task 7	44
5.8.1	Goal Description:	44
5.8.2	Assumptions:	44
5.8.3	Implementation Approach:	44
5.8.4	Interface Specifications:	45
5.8.5	Results:	45
5.9	Task 8	46
5.9.1	Goal Description:	46
5.9.2	Assumptions:	46
5.9.3	Implementation Approach:	46
5.9.4	Interface Specifications:	47
5.9.5	Results:	47
5.10	Task 9	50
5.10.1	Goal Description:	50

5.10.2	Assumptions:	50
5.10.3	Implementation Approach:	50
5.10.4	Interface Specifications:	51
5.10.5	Results:	51
5.11	Task 10	54
5.11.1	Goal Description:	54
5.11.2	Assumptions:	55
5.11.3	Implementation Approach:	55
5.11.4	Interface Specifications:	55
5.11.5	Results:	56
5.12	Task 11	57
5.12.1	Goal Description:	57
5.12.2	Assumptions:	57
5.12.3	Implementation Approach:	58
5.12.4	Interface Specifications:	59
5.12.5	Results:	59
5.12.6	Key Observations:	74
6	System Requirements & Execution Instructions	75
7	Related Works	76
8	Conclusions	77
A	Specific Roles and Details of the Group Members	80

1 Abstract

In our Phase 2 Project Report, we discuss the research, implementations, and results of Dimensionality Reduction techniques on the Caltech101 Dataset. In general, the core of the tasks (from 0 to 10) utilizes the feature space models from Phase 1, create and use dimensionality reduction techniques such as Singular Value Decomposition (SVD) & Non-Negative Matrix Factorization (NNMF), and generate latent semantics (based on these dimensionality reduction techniques) in order to provide the top-k similar images or labels based on given images or labels. Additionally, we discuss our results of developing the personalized Page Rank algorithm (task 11). In this phase, we place a large emphasis on making our code repository modular and extendable (enabling code reusability) from Phase 1 and ensuring our design allows us to utilize it in future phases. Additionally, we utilized various math formulas to solve the tasks and Jupyter Notebooks to visualize the images with an easy-to-use interface for each task.

2 Keywords

feature descriptor, image processing, vector models, curse of dimensionality, dimensionality reduction, page rank, feature extraction, similarity measure, distance measure, Python, ResNet, Jupyter Notebook

3 Structure of Report

Following this section, we introduce the relevant mathematical formulae (and key implementation details surrounding the formulae) used for the specific tasks in this Phase. Suitable use cases of these mathematical formulae during the tasks will be cited back to this section to ensure the report remains easily readable. Next, we will systematically discuss the goal description, assumptions (if any), implementation approach, and results (if any) for each Task described in the Specification Outline. Following this, we will go through separate sections on System Requirements & Execution Instructions to run the code base provided (alternatively, you can refer to the README.md file described in the primary code base for these details). The report will continue with separate sections on Related Works, Overall Conclusions, and notable References used to complete the tasks. The report will conclude

with an Appendix detailing the efforts of each group member to complete this Phase-wise development.

IMPORTANT NOTE: Any Missing Outputs in this Report can be found in the Outputs Folder attached with this Submission.

4 Relevant Formulas and Corresponding Implementations

1. **Distance Measures:** We implement 4 different Distance Measures for our experiments under Task 3. Distance Measures are either pre-computed or implemented using the Scipy Spatial Distance module [1]:

$$\begin{aligned}\text{Cosine Distance} &= 1 - \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \\ \text{Cosine Similarity} &= \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} \\ \text{Correlation Distance} &= 1 - \frac{(\mathbf{u} - \mathbf{u}_{\text{mean}}) \cdot (\mathbf{v} - \mathbf{v}_{\text{mean}})}{\|(\mathbf{u} - \mathbf{u}_{\text{mean}})\|_2 \cdot \|(\mathbf{v} - \mathbf{v}_{\text{mean}})\|_2} \\ \text{Manhattan Distance} &= \sum_{i=1}^n |A_i - B_i| \\ \text{Euclidean Distance} &= \sum_{i=1}^n \sqrt{(A_i^2 - B_i^2)}\end{aligned}$$

Distance functions for latent semantics were chosen based on subjective analysis of the results. The following is a list of all latent / feature space combinations for each task, and what distance were used for each. Note that if any combination is not mentioned, the distance used for it is the same as the distance used for the feature space.

Manhattan distance (City-block distance): This distance can be imagined as the length needed to move between two points in a grid where you can only move up, down, left or right. This measure is sensitive to differences along each dimension of the feature vectors, capturing the total absolute difference

between corresponding moments, which makes it great for assessing differences in color distribution in terms of absolute deviations. Since it does not square the differences like Euclidean distance, it is less susceptible outliers when many dimensions differ.

Euclidean Distance: We used Euclidean distances, when we want to give more importance to the maximum difference between two feature spaces. This would be ideally used in feature spaces where the cluster separation as well as cluster tightness is smaller.

Correlation similarity: This measures the linear relationship between HOG feature vectors, and performs well with features that have a strong linear correlation or dependency in their gradient orientation distributions, taking into account directionality.

Cosine distance: We use Cosine since the output of ResNet layers are normalized, reducing the importance of magnitude. It is good at discerning semantic or structural similarity rather than their absolute feature values. It also works well with sparse feature spaces such as those found in CNNs.

Cosine similarity is used when we try to construct label_label or image_image similarity matrices. It is good at discerning semantic or structural similarity rather than their absolute feature values.

```
"color": cityblock
"t7LS1_color": correlation
"t8LS1_color": cityblock
"t9LS1_color": cityblock
"t10LS1_color": euclidean
"hog": correlation
"t7LS2_hog": cosine
"t8LS2_hog": cosine
"t9LS2_hog": cosine
"t10LS2_hog": correlation
"avgpool": cosine
```

"t7LS3_avgpool": cosine
"t8LS3_avgpool": cosine
"t9LS3_avgpool": cosine
"t10LS3_avgpool": cosine
"layer3": cosine
"t7LS4_layer3": cosine
"t8LS4_layer3": cosine
"t9LS4_layer3": cosine
"t10LS4_layer3": cosine
"fc": cosine
"resnet": cosine
"t7LS1_resnet": euclidean
"t8LS1_resnet": euclidean
"t9LS1_resnet": cosine
"t10LS1_resnet": euclidean
"t7LS2_resnet": cityblock
"t8LS2_resnet": cityblock
"t9LS2_resnet": cosine
"t10LS2_resnet": cityblock
"t7LS3_resnet": correlation
"t8LS3_resnet": correlation
"t9LS3_resnet": cosine
"t10LS3_resnet": cosine
"t7LS4_resnet": cosine
"t8LS4_resnet": cosine
"t9LS4_resnet": cosine
"t10LS4_resnet": cosine
"label_label_similarity": cosine_similarity
"image_image_similarity": cosine_similarity

2. **Clustering Algorithms:** We primarily experiment with two popular clustering algorithm: K-Means Clustering [2] & DBScan [3]. This was widely used to find label-similarity metrics for relevant tasks in this Phase. K-Means Clustering was implemented from scratch using relevant matrix manipulation & distance measure functionality using NumPy, SciPy, and Scikit-learn. We implemented DBScan using in-built functionality. For all purposes in this project, we mainly use the K-Means Clustering as our primary Clustering algorithm choice; however, for the sake of consistency, we provide the formula for DBScan as well.

(a) **K-Means Clustering:**

$$J = \sum_{i=1}^K \sum_{j=1}^n ||x_j^{(i)} - \mu_i||^2$$

where, J = Objective Function to be Minimized

K = Number of Clusters

n = Number of Data Points

$x_j^{(i)}$ = j-th data point in cluster i

μ_i = centroid of cluster i

(b) **DBScan (Density-Based Spatial Clustering):**

$$\text{Core}(p, \varepsilon, \text{MinPts}) = |\{q \in D : \text{dist}(p, q) \leq \varepsilon\}| \geq \text{MinPts}$$

$$\text{Border}(p, \varepsilon, \text{MinPts}) = |\{q \in D : \text{dist}(p, q) \leq \varepsilon\}| < \text{MinPts} \text{ and}$$

$$\exists o \in D : \text{Core}(o, \varepsilon, \text{MinPts}) \text{ where } \text{dist}(p, o) \leq \varepsilon$$

$$\text{Noise}(p) = \neg(\text{Core}(p, \varepsilon, \text{MinPts}) \vee \text{Border}(p, \varepsilon, \text{MinPts}))$$

$$\text{Clusters, } C = \{p : \text{Core}(p, \varepsilon, \text{MinPts})\}$$

ε = the maximum distance to be considered a neighbor

MinPts = minimum number of data points required within distance ε

3. **Dimensionality Reduction Algorithms:** A primary theme of this phase of the Project is to explore the curse of Dimensionality when we create the Image-Image or Label-Label Similarity Matrices. To work around the Curse of Dimensionality, we implement the following Dimensionality Reduction algorithms and explore their effects on the

accuracy of image-image & label-label similarity (or distance) across the Specification Tasks.

(a) **Singular Value Decomposition (SVD):**

$$A = U\Sigma V^T$$

where, A = Original Matrix

U = left singular vectors matrix, an $m \times m$ orthogonal matrix

Σ = diagonal singular values matrix, an $m \times n$ diagonal matrix

V^T = transpose of right singular vectors matrix, an $n \times n$ orthogonal matrix

SVD is implemented from scratch in our codebase, with the help of several numpy functions (especially the linear algebra module within numpy to calculate the Eigen Value Matrix Σ) [4].

(b) **Non-Negative Matrix Factorization (NNMF):**

$$V \approx WH$$

where, V = original non-negative matrix with dimensions $m \times n$

W = non-negative factor matrix with dimensions $m \times r$.

H = non-negative coefficient matrix with dimensions $r \times n$.

$$\text{Objective Function : } \min_{W, H} \frac{1}{2} \|V - WH\|_F^2$$

$$W \geq 0, \quad H \geq 0$$

where, $\|\cdot\|_F$ = represents Frobenius Norm Value.

NNMF [5] is implemented from scratch on our codebase, with the help of numpy for matrix calculations. We pre-define an **epoch count of 200** to gradually reduce the Input Matrix to the K dimensions for all usages of NNMF in this project phase.

(c) **Latent Dirichlet Allocation (LDA):** The generative process

for Latent Dirichlet Allocation [6] is as follows:

For each topic $k = 1, \dots, K$:

$$\theta_k \sim \text{Dir}(\alpha)$$

For each document $d = 1, \dots, D$:

$$\theta_d \sim \text{Dir}(\beta)$$

For each word $n = 1, \dots, N$:

$$z_{d,n} \sim \text{Multinomial}(\theta_d)$$

$$w_{d,n} \sim \text{Multinomial}(\theta_{z_{d,n}})$$

θ_k = distribution of topics in the dataset

K = total number of topics

α = Dirichlet prior for the topic distribution

θ_d = distribution of topics in document d

D = total number of documents

$z_{d,n}$ = topic assignment for word n in document d

$w_{d,n}$ = observed word for topic assignment $z_{d,n}$

Latent Dirichlet Allocation (LDA) is implemented with the direct usage of Scikit-Learn's LDA Decomposition Module.

(d) **Candecomp Parafac (CP) Decomposition:**

$$X \approx \sum_{r=1}^R A_r \circ B_r \circ C_r$$

where, X = represents the original tensor for decompose

A_r, B_r, C_r = factor matrices for mode-1, mode-2, and mode-3

r = rank of the decomposition

$\circ$ = matrix outer product

CP Decomposition [7] is implemented with the usage of the parafac decomposition functionality in the tensorly python package.

(e) **KMeans:** We implement a dimensionality reduction technique that leverages KMeans clustering as described earlier. The idea is that we compute K clusters, and the new latent space is the distance of a query vector to each cluster's centroid.

4. **Personalized Page Rank Algorithm:** The Personalized PageRank algorithm [8] computes a personalized PageRank score for a given node or set of nodes in a graph.

$$p(t+1) = (1 - \alpha) * M * p(t) + \alpha * p_0$$

$$p(t+1) = \text{updated PageRank}$$

$$p(t) = \text{vector of PageRank scores at time } t$$

$$\alpha = \text{teleport probability}$$

$$M = \text{transition matrix of the graph}$$

$$p_0 = \text{personalized node or seed node's probability vector}$$

5 The Tasks

5.1 Task 0

5.1.1 Task 0a - Goal Description:

Using pre-trained RESNET50 neural network model, map even-numbered (labelled) images in the Caltech101 data set into 5 different feature spaces and store the resulting data vectors:

1. Color moments, CM10x10
2. Histograms of oriented gradients, HOG
3. ResNet-AvgPool-1024
4. ResNet-Layer3-1024
5. ResNet-FC-1000

In the database, store not only the Image IDs and feature vectors, but also the original image labels.

In addition to the above, we compute representative vectors for each label under each feature space. This is a time consuming process, so in order to save time, we compute it during this task, which we treat sort of as a "database generation" task.

5.1.2 Task 0a - Assumptions Made:

1. For task 0a, we directly reuse the code we used for Phase 1. Specifically, we have separate modules for each of the relevant code (feature descriptor calculation & distance measures) to ensure reusability for now and future phases of the project.
2. A critical assumption is that 0a represents a mandatory step for generating the relevant feature descriptors and representative label vectors, which we use as a basis for the rest of this project.

5.1.3 Task 0a - Implementation Approach:

1. The python script named task_0a.py represents the code file for this task.
2. Along with the five feature descriptors defined above, we also extract the overall ResNet50 Model Output (which is required for task 2b).
3. The formulae/methodology used to calculate each of the Feature Descriptors can be found in our reports for Phase 1, but we specify it here anyway:

Color moments: vec: [`<cell_1 <red <mean, stdev, skew>>, <green <...>>, <blue <...>>>, <cell_2 ...>...`]

HOG: vec: [`<cell_1 <0, 40, 80, 120 ... 320>>, <cell_2 ...>...`]

avgpool: Shape of ResNet avgpool is 1x2048, we average every 2 values sequentially to obtain a 1024 dimension vector.

layer3: Resnet's layer 3 shape is 1x1024x14x14, We take the mean of each 14x14 slice, resulting in a 1024 dimension vector.

fc: Resnet's fc layer output.

resnet: Resnet model's final output (after applying SoftMax).

4. We compute representative vectors for each label under each feature space as follows:
 - (a) Consider all vectors for a particular label, cluster them into 5 clusters using KMeans.

- (b) Capture the cluster centroids for each label, and store in a dictionary where the key is the label.

5.1.4 Task 0a - Results:

There is no specific input to be provided for this task. Hence, there are no direct results. However, we create and store separate files for each of the feature descriptors, with respect to the image ID requirement mentioned, which combines to create the overall Feature Descriptor Database. This database, we assume, will be used throughout the rest of the project. These are stored in the directory titled "database" within the codebase.

As an example, here is the output for IMG ID 0 for Color moments (truncated):

```
0 -> ('Faces', tensor([100.7500, 41.3330, 22.4105, 99.2667, 45.0242,
25.0887, 93.9633, 46.9503, 23.0422, 130.4167, 25.7842, -20.5028,
134.4833, 26.3796, -20.8315, 128.3133, 24.9557, -19.8871, 123.7467,
37.4010, -36.5426, 128.2567, 37.8975, -35.9094, 121.0633, 35.9962,
-33.7913, 96.0700, 62.6277, 26.3150, 100.6967, 64.4597, 27.0212,
100.7300, 61.8235, 12.5499, 159.6167, ...
```

Here is an example of the representative vectors found for LABEL Faces under Color Moments feature space (truncated):

```
Faces -> [array([ 1.64966019e+02, 2.40975151e+01, -1.12671165e+01,
1.67978210e+02, 2.63620453e+01, -1.15305939e+01, 1.60023468e+02,
2.84955788e+01, ... array([ 7.49864655e+01, 1.32117233e+01, -1.93036509e+00,
7.41158142e+01, 1.27037315e+01, 7.95774460e-02, 6.38545074e+01, 1.20763302e+01,
... 5 vectors in total.
```

5.1.5 Task 0b - Goal Description:

Implement a program which, given (a) an (even or odd numbered) imageID or an image file, (b) a user selected feature space, and (c) positive integer k, identifies and visualizes the most similar k images, along with their scores, under the selected feature space.

5.1.6 Task 0b - Assumptions Made:

1. We can pre-configure the relevant distance measure for each feature descriptor based on the best accuracy we see across the image dataset.

5.1.7 Task 0b - Implementation Approach:

1. The python notebook named task_0b.ipynb represents the descriptive notebook to run this task.
2. We first extract the feature descriptor (chosen by the user separately) for the input image.
3. Second, we load the particular feature descriptor database file, and find the distance scores between the input feature descriptor and each feature descriptor from the database and rank the top k matches in ascending order.
4. The relevant top-ranked images are displayed, and the distance score is mentioned.

5.1.8 Task 0b - Interface Specifications:

1. First, provide a valid input ID with respect to the CalTech101 Dataset or provide a direct input image path.
2. Second, choose a valid Feature Descriptor for the comparison.
3. Finally, provide a valid K value representing the total number of top comparisons required.

5.1.9 Task 0b - Results:

As illustrated by the outputs, we get similar images to a given image based on a selected feature space. For few input images the outputs are not ideal, this is because of the distance functions that we selected. The distance functions were selected to maximize the number of similar images to have similar characteristics.

Results for Image ID: 2500 - Feature Descriptor: ResNet50 Fully Connected Layer



5.2 Task 1

5.2.1 Goal Description:

Implement a program which, given (a) a query label, l (b) a user selected feature space, and (c) positive integer k, identifies and visualizes the most relevant k images for the given label l, along with their scores, under the selected feature space.

5.2.2 Assumptions Made: N/A

5.2.3 Implementation Approach:

1. Representative Feature Descriptor for Each Label: This is calculated by finding the **centroid vector of all image feature descriptors** filtered for the input label.
2. Using this centroid vector as the primary comparison value, we sort and find the top k images with the closest distances (using the pre-configured distance scores for each feature descriptor, similar to Task 0b) and visualize the corresponding images as output.

5.2.4 Interface Specifications:

1. First, provide a valid label name OR label ID with respect to the Cal-Tech101 Dataset.
2. Second, choose a valid Feature Descriptor for the comparison.
3. Finally, provide a valid K value representing the total number of top comparisons required.

5.2.5 Results:

Results for Label ID: 0 - Feature Descriptor: Color Moments

Results for Label ID: 0 - Feature Descriptor: Histogram of Oriented Gradients (HOG)

Results for Label ID: 0 - Feature Descriptor: ResNet50 L3 Layer

Results for Label ID: 0 - Feature Descriptor: ResNet50 Avg-Pooling Layer

Results for Label ID: 0 - Feature Descriptor: ResNet50 Fully Connected (FC) Layer

Results for Label ID: 20 - Feature Descriptor: Color Moments
Results for Label ID: 20 - Feature Descriptor: Histogram of Oriented Gradients (HOG)
Results for Label ID: 20 - Feature Descriptor: ResNet50 L3 Layer
Results for Label ID: 20 - Feature Descriptor: ResNet50 Avg-Pooling Layer
Results for Label ID: 20 - Feature Descriptor: ResNet50 Fully Connected (FC) Layer

Results for Label ID: 55 - Feature Descriptor: Color Moments
Results for Label ID: 55 - Feature Descriptor: Histogram of Oriented Gradients (HOG)
Results for Label ID: 55 - Feature Descriptor: ResNet50 L3 Layer
Results for Label ID: 55 - Feature Descriptor: ResNet50 Avg-Pooling Layer
Results for Label ID: 55 - Feature Descriptor: ResNet50 Fully Connected (FC) Layer

Results for Label ID: 100 - Feature Descriptor: Color Moments
Results for Label ID: 100 - Feature Descriptor: Histogram of Oriented Gradients (HOG)
Results for Label ID: 100 - Feature Descriptor: ResNet50 L3 Layer
Results for Label ID: 100 - Feature Descriptor: ResNet50 Avg-Pooling Layer
Results for Label ID: 100 - Feature Descriptor: ResNet50 Fully Connected (FC) Layer

5.3 Task 2

5.3.1 Task 2a - Goal Description:

Implement a program which, given (a) a query imageID or image file, (b) a user selected feature space, and (c) positive integer k, identifies and lists k most likely matching labels, along with their scores, under the selected feature space.

5.3.2 Task 2a - Assumptions Made:

1. We assume that each of the likely labels to be output must be unique.

5.3.3 Task 2a - Implementation Approach:

1. The python notebook named task_2a.ipynb represents the descriptive notebook to run this task.
2. We first extract the feature descriptor (chosen by the user separately) for the input image.
3. Second, we load the particular feature descriptor database file and find the sorted distance scores between the input feature descriptor and each feature descriptor from the database.
4. **Label Filtering:** To rank the top k unique labels, we iterate through our sorted distance scores and create a ranking for each label based on **best-match** score.
5. We opted for the best-match score criteria as it provided a good accuracy for the test input images. We also conducted experiments using **best-mean score** per label, but do not include the results in this report.
6. The relevant top-ranked labels are displayed, and their respective closest distance score is mentioned.

5.3.4 Task 2a - Interface Specifications:

1. First, provide a valid input ID with respect to the CalTech101 Dataset OR provide a direct input image path.

2. Second, choose a valid Feature Descriptor for the comparison.
3. Finally, provide a valid K value representing the total number of top comparisons required.

5.3.5 Task 2a - Results:

5.3.6 Task 2b - Goal Description:

Implement a program which, given (a) a query imageID or image file and (b) positive integer k, identifies and lists k most likely matching labels, along with their scores, under the RESNET50 neural network model.

5.3.7 Task 2b - Assumptions Made:

1. We assume that each of the likely labels to be output must be unique.

5.3.8 Task 2b - Implementation Approach:

1. The python notebook named task_2b.ipynb represents the descriptive notebook to run this task.
2. First, we load the ResNet50 Output feature descriptor database file and find the sorted distance scores between the input feature descriptor and each feature descriptor from the database.
3. **Label Filtering:** We use a label-filtering algorithm similar to Task 2a (along with the same justifications) to retrieve the top k-ranked labels.
4. The relevant top-ranked labels are displayed, and their respective closest distance score is mentioned.

5.3.9 Task 2b - Interface Specifications:

1. First, provide a valid input ID with respect to the CalTech101 Dataset OR provide a direct input image path.
2. Second, provide a valid K value representing the total number of top comparisons required with respect to the overall ResNet50 Output features.

5.3.10 Task 2b - Results:

As seen in the results below, we get similar labels based on the input images. We do get a few incorrect labels in our results like in case of Faces we get Faces and Faces_easy and we get some random labels. This could be because in this case, no other label resembles a face and hence we start getting random labels in the output.

a. id: 0, k = 10



K = 10 most similar labels by resnet feature space using distance: cosine

1.	Label: Faces	Distance: 0.19083738327026367
2.	Label: Faces_easy	Distance: 0.3079897165298462
3.	Label: dollar_bill	Distance: 0.5836428105831146
4.	Label: scissors	Distance: 0.6451184749603271
5.	Label: wrench	Distance: 0.6593480110168457
6.	Label: yin_yang	Distance: 0.6739566326141357
7.	Label: stop_sign	Distance: 0.6774144172668457
8.	Label: camera	Distance: 0.6799745857715607
9.	Label: stapler	Distance: 0.6835103929042816
10.	Label: garfield	Distance: 0.6912636756896973

id: 880, k = 10



K = 10 most similar labels by resnet feature space using distance: cosine

1.	Label: Leopards	Distance: 0.07137942314147949
2.	Label: wild_cat	Distance: 0.3807564377784729
3.	Label: brontosaurus	Distance: 0.5894375443458557
4.	Label: cougar_face	Distance: 0.675415962934494
5.	Label: cougar_body	Distance: 0.6939966082572937
6.	Label: platypus	Distance: 0.7239813506603241
7.	Label: starfish	Distance: 0.7478203177452087
8.	Label: gerenuk	Distance: 0.7538588047027588
9.	Label: dollar_bill	Distance: 0.7565493285655975
10.	Label: stegosaurus	Distance: 0.7621845304965973

c. id: 2500, k = 10



K = 10 most similar labels by resnet feature space using distance: cosine

1.	Label: airplanes	Distance: 0.050704240798950195
2.	Label: helicopter	Distance: 0.34586799144744873
3.	Label: ferry	Distance: 0.7044163644313812
4.	Label: brontosaurus	Distance: 0.746015727519989
5.	Label: metronome	Distance: 0.7677012085914612
6.	Label: minaret	Distance: 0.7800235003232956
7.	Label: dolphin	Distance: 0.809442013502121
8.	Label: scorpion	Distance: 0.8104687333106995
9.	Label: pagoda	Distance: 0.8134507089853287
10.	Label: elephant	Distance: 0.8235170990228653

d. id: 5122, k = 10



K = 10 most similar labels by resnet feature space using distance: cosine

1.	Label: garfield	Distance: 0.10975176095962524
2.	Label: snoopy	Distance: 0.21259862184524536
3.	Label: bass	Distance: 0.25997382402420044
4.	Label: cannon	Distance: 0.2685782313346863
5.	Label: octopus	Distance: 0.3146929144859314
6.	Label: mayfly	Distance: 0.3194388151168823
7.	Label: ant	Distance: 0.35071778297424316
8.	Label: lobster	Distance: 0.3793313503265381
9.	Label: cup	Distance: 0.3855140209197998
10.	Label: brain	Distance: 0.3866146206855774

e. id: 8676, k = 10



```
K = 10 most similar labels by resnet feature space using distance: cosine
1. Label: yin_yang Distance: 0.3299492597579956
2. Label: headphone Distance: 0.5289261639118195
3. Label: soccer_ball Distance: 0.5513787269592285
4. Label: camera Distance: 0.6145080327987671
5. Label: gramophone Distance: 0.6573273241519928
6. Label: stop_sign Distance: 0.6660142838954926
7. Label: scissors Distance: 0.6795521676540375
8. Label: cellphone Distance: 0.6813301146030426
9. Label: nautilus Distance: 0.6991804540157318
10. Label: metronome Distance: 0.7016085386276245
```

5.4 Task 3 - Latent Semantics 1 (LS1)

5.4.1 Goal Description:

Implement a program which (a) given one of the feature models, (b) a user specified value of k, (c) one of the four dimensionality reduction techniques (SVD, NNMF, LDA, k-means) chosen by the user, reports the top-k latent semantics extracted under the selected feature space.

1. Store the latent semantics in a properly named output file,
2. List imageID-weight pairs, ordered in decreasing order of weights.

5.4.2 Assumptions:

1. We can use libraries for LDA, but not the other 3 feature reduction techniques.
2. We can use libraries for Eigen decomposition.

5.4.3 Implementation Approach:

1. We define the concept of a feature reducer. A feature reducer is an object that takes an input dataset of vectors, and returns the dataset

reduces to K dimensions. It also provides the same functionality for a query vector, reducing it to K dimensions.

2. Singular Valued Decomposition:

- (a) Derive U , S and V matrices for the provided database vectors through Eigen decomposition.
- (b) For a query vector, return the reduced feature vector by multiplying it with V .
- (c) You can reconstruct the original database via $U \text{ dot } S \text{ dot } V \text{ transpose}$.

3. Non-negative Matrix Factorization:

- (a) We scale the input database.
- (b) We compute W and H matrices through 200 iterations starting from random assignment.
- (c) For a query vector, we find W again by creating a database from the query vector.

4. Latent Dirichlet Allocation:

- (a) We normalize the input database.
- (b) We use the `LatentDirichletAllocation` implementation provided by Sklearn.

5. KMeans:

- (a) We scale the input database.
- (b) We choose K random centroids.
- (c) We iterate 300 times, in each iteration, binning each vector to a cluster based on the closest centroid.
- (d) We recompute the centroids in each iteration.
- (e) Given a query vector, we compute its distance to each centroid, which produces a K length vector.

6. We display sorted weights such that for each latent dimension in K , we show the vectors (image ID) of the image ordered by the highest value in that dimension.

5.4.4 Interface Specifications:

1. To run this program, run all the cells in the task_3.ipynb file. This task depends on task 0 to be run successfully
2. The program will prompt to provide a valid feature space in the format (color, hog, avgpool, layer3, fc, resnet)
3. After this the program will prompt for a dimensionality reduction technique as an input (svd, nmmf, lda, kmeans)
4. It will also provide a prompt for a valid K value representing the number latent features that the input feature would be reduced to.

5.4.5 Results:

The label weight results are generated in the following format:

Image ID - weight pairs sorted in descending order of weights for each latent feature:

Latent feature: 1

(ID: 800 , Weight: 0.053155404), (ID: 812 , Weight: 0.052846022),
(ID: 32 , Weight: 0.052470617), (ID: 230 , Weight: 0.052324735),
(ID: 16 , Weight: 0.052250296), ...

5.5 Task 4 - Latent Semantics 2 (LS2)

5.5.1 Goal Description:

Implement a program which (a) given one of the feature models, (b) a user specified value of k , reports the top- k latent semantics extracted using CP-decomposition of a three modal (image-feature-label) tensor under the selected features space. Each latent semantic should be presented in the form of a list of label-weight pairs, ordered in decreasing order of weights.

1. Store the latent semantics in a properly named output file,
2. List label-weight pairs, ordered in decreasing order of weights.

5.5.2 Assumptions:

1. We assumed that the CP decomposition would work with it being in the image-label-feature format as it would be easier to calculate (for every image we take the distance from each representative image of the label)
2. A lower reconstruction accuracy would not hinder the performance of subsequent tasks depending on task 4

5.5.3 Implementation Approach:

1. As an input for the CP decomposition we created the tensor in image-label-feature format.
2. For this, we found representative labels for each label in the given feature space. Then for each image row in the tensor we created a Number of Labels (101) x Number of Features (for the selected latent) matrix. In this 2D matrix, we populated zeroes for every label row except for the label row where the image belongs. This results in a Number of Images x Number of Labels x Number of Features 3D tensor.
3. Our implementation uses the parafac method from the tensorly library and the cp_als_rand_init method from the tensorlearn library for the finding the CP-decomposition of the matrix.
4. On running these two CP decompositions we got similar results. The reconstruction accuracy was roughly around 20%.

5. Normalization of the inputs didn't have too much of an impact on the reconstruction accuracy
6. We also tried finding the distance between each image to the representative image of each label to populate the tensor using a combination of different distance functions. However, even this approach didn't give us a better accuracy and instead took a much longer time. Hence, we have not included this in our final code submission

5.5.4 Interface Specifications:

1. To run this program, run all the cells in the task_4.ipynb file. This task depends on task 0 to be run successfully
2. The program will prompt to provide a valid feature space in the format (color, hog, avgpool, layer3, fc, resnet)
3. It will also provide a prompt for a valid K value representing the number of latent features that the input feature would be reduced to.

5.5.5 Results:

The output to the program (label weight pairs) specified is in the following format: (for the input K=5 and feature space of resnet_output)

Latent feature: 1

(Label: Faces , Weight: 0.9877737377314768), (Label: Faces_easy , Weight: 0.15589433295661465), for all labels

Latent feature: 2

(Label: Faces_easy , Weight: 0.9377029661950198), (Label: Faces , Weight: 0.3474379760317825),, for all labels

We observed low reconstruction accuracy while finding the CP decomposition. On experimenting with the libraries, number of iterations and the input tensor format, we didn't find significantly better results. Running the CP decomposition also takes an exceptionally large amount of time (approx 30 seconds per iteration), and we read certain literature regarding the ineffectiveness of using more than 100 iterations in CP decomposition [9]. We expected this to adversely affect the other algorithms which depend on this latent semantic, however, this does not seem to be the case. This could be because the latent features that are learnt are so distinct from each other that

the algorithms are able to distinguish between the latent spaces of different labels. However, this does not mean that the features that are being learnt are accurate.

5.6 Task 5 - Latent Semantics (LS3)

5.6.1 Goal Description:

Implement a program which, (a) given one of the feature models and (b) a value k ,

1. creates (and saves) a label-label similarity matrix,
2. performs a user selected dimensionality reduction technique (SVD, NNMF, LDA, k-means) on this label-label similarity matrix,
3. stores the latent semantics in a properly named output file,
4. lists label-weight pairs, ordered in decreasing order of weights.

5.6.2 Assumptions:

1. We assume that taking a mean of the feature space for a particular label would give us its representative image

5.6.3 Implementation Approach:

1. To create the label-label similarity matrix, we first found the representative images of each label. This was done by taking a mean of the feature space of a particular label.
2. After this we constructed the label-label similarity matrix by finding the cosine similarity score between each representative vector and placing it at the appropriate spot in the matrix. This would create a 101 x 101 symmetric matrix (total number of labels are 101 and since cosine similarity is associative).
3. Lastly, after creating the label-label similarity matrix, we then feed it along with the number of components in the latent space (K) into our reducer classes based on the user selected dimensionality reduction technique. This would give us a 101 x K latent feature matrix. (for more information about the reducer, refer to implementation details in task 3)

5.6.4 Interface Specifications:

1. To run this program, run all the cells in the task_5.ipynb file. This task depends on task 0 to be run successfully
2. The program will prompt to provide a valid feature space in the format (color, hog, avgpool, layer3, fc, resnet)
3. After this the program will prompt for a dimensionality reduction technique as an input (svd, nnmf, lda, kmeans)
4. It will also provide a prompt for a valid K value representing the number latent features that the input feature would be reduced to.

5.6.5 Results:

The output to the program (label weight pairs) specified is in the following format: (for the input K=5, latent semantics resnet_output and the kmeans feature reduction technique)

```
Latent feature: 1
(Label: octopus , Weight: 20.01717559366013), (Label: menorah
, Weight: 17.069155374846254), .... for all labels
Latent feature: 2
(Label: emu , Weight: 15.64015224658079), (Label: joshua_tree
, Weight: 15.454988259617968),, .... for all labels
```

We experimented with various similarity metrics like cosine, correlation and intersection. We selected cosine similarity to do the label to label comparison because it provided a better accuracy in the later tasks. This could be because cosine similarity depends on the angle of the vectors rather than the magnitudes which suits the feature space of our data. We also picked cosine similarity due to its performance in higher dimensional spaces and sparse data [10].

5.7 Task 6 - Latent Semantics (LS4)

5.7.1 Goal Description:

Implement a program which, (a) given one of the feature models and (b) a value k ,

1. creates (and saves) an image-image similarity matrix,
2. performs a user selected dimensionality reduction technique (SVD, NNMF, LDA, k-means) on this image-image similarity matrix,
3. stores the latent semantics in a properly named output file,
4. lists image-weight pairs, ordered in decreasing order of weights.

5.7.2 Assumptions:

1. This task expects that Task_0 (task_0a.py) has been executed.
2. The code takes about 6 minutes to run since the image x image matrix has to be generated. This is be around $4339 \text{ (number of images)} \times 4339 \text{ (number of images)} = 18,826,921$ iterations. Since this remains constant for a particular feature space, we store it for that feature space

5.7.3 Implementation Approach:

1. As an input, we have the image-id feature vector matrix for one of the feature models.
2. For each image in the feature model database, we iterate through the database, compare the similarity (cosine similarity) and store the results in a dictionary. Layout of the dictionary is as follows: $\{\text{'image_id': (image_label, [}d_1, d_2, \dots, d_n\text{])}, \dots\}$. Note: As for the generation of the image-image similarity matrix, the generated matrix is stored as a database.
3. Using one of the dimensionality reduction techniques, generate the reducer. The reducer is used to generate a latent semantics database for specific combinations of feature models, dimensionality reduction techniques and no of dimensions in reduced space. The reducer will also be used in future tasks to produce latent dimensions of a query image. (for

more information about the reducer, refer to implementation details in task 3)

4. Lastly, it produces a similarity matrix which defines the images in terms of reduced dimensions. For each of the latent dimensions, the weights are sorted in decreasing order, and (image-id weight) pairs are printed.

5.7.4 Interface Specifications:

1. Provide one of the valid feature models (color, hog, avgpool, layer3, fc, resnet).
2. Next, provide one of the dimensionality reduction techniques to be used - 'svd' [Singular Value Decomposition], 'nnmf' [Non-Negative Matrix Factorization], 'lda' [Latent Dirichlet Allocation], 'kmeans' [K-Means]
3. Finally, provide K, required no of reduced dimensions for feature vector

5.7.5 Results:

Below are some of the outputs that we tested the data on. Based on our testing we found that in the subsequent tasks our approach performs well. We tried cutting down the feature storage size by using compression techniques and successfully implemented it. We also tried to reduce the time taken to run the task but could not optimize it too much since the number of comparisons made by the data was quite large to begin with.

Latent Semantics & Image-Weight Pairs: Feature Descriptor: L3, Dimensionality Reduction: SVD, K=5

Task 6, Config 1: Layer3, SVD, K=5, Latent Semantics:

Top K latent semantics:

```
[[-5.81234836e+01 -1.29448218e+00 -1.04980468e+00 -3.24646781e-01
  2.22273248e-01]
 [-5.78402578e+01 -1.40159763e+00 -7.18405712e-01 -3.20406906e-01
  -1.62832374e-01]
 [-5.83740084e+01 -1.42373779e+00 -8.92252358e-01 -2.88464972e-01
  2.37042543e-01]
 ...
 [-5.35629635e+01  8.24918411e-01 -6.40104458e-01  1.69704489e-01
  5.72635517e-02]
 [-5.70250858e+01  4.02968683e-01 -3.19581738e-01  5.88126227e-01
  3.95828063e-01]
 [-5.64422934e+01  1.10913736e+00 -5.07284459e-01  5.38171589e-01
  1.27165949e-01]]
Shape: (4339, 5)
```

Task 6, Config 1: Layer3, SVD, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 7530 , Weight: -0.012364452823871941), (ID: 7548 , Weight: -0.012822454439594146), (ID: 7524 , Weight:

Latent feature: 2
(ID: 7326 , Weight: 0.03971642091614633), (ID: 5112 , Weight: 0.03660125154962249), (ID: 8600 , Weight:

Latent feature: 3
(ID: 2434 , Weight: 0.030048400799792), (ID: 3494 , Weight: 0.029660251651826588), (ID: 2070 , Weight:

Latent feature: 4
(ID: 7420 , Weight: 0.036012873465400184), (ID: 8012 , Weight: 0.03144502766313733), (ID: 7398 , Weight:

Latent feature: 5
(ID: 1918 , Weight: 0.05372371842758053), (ID: 6052 , Weight: 0.05142726189238589), (ID: 2832 , Weight:
```

■

Latent Semantics & Image-Weight Pairs: Feature Descriptor: L3, Dimensionality Reduction: NNMF, K=5

Task 6, Config 2: Layer3, NNMF, K=5, Latent Semantics:

```
Top K latent semantics:
[[0.36385542 0.27837111 0.71867453 1.09290544 2.02785502]
 [0.12469737 0.60109307 0.63114084 1.09964829 1.89747267]
 [0.46650424 0.40883282 0.624086 1.02803942 2.06254056]
 ...
 [0.84610906 0.09697641 0.74467551 0.51814934 0.47011143]
 [1.22438915 0.56052544 1.12080189 0.32991096 0.87194211]
 [0.70923604 0.47567615 1.5893676 0.61647628 0.45818349]]
Shape: (4339, 5)
```

Task 6, Config 2: Layer3, NNMF, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 1440 , Weight: 2.759433491880106), (ID: 8606 , Weight: 2.7136949737485865), (ID: 6538 , Weight:

Latent feature: 2
(ID: 6694 , Weight: 1.8898230090399257), (ID: 2098 , Weight: 1.8823305855124564), (ID: 5674 , Weight:

Latent feature: 3
(ID: 5008 , Weight: 2.289220221082965), (ID: 5000 , Weight: 2.060958962182286), (ID: 6342 , Weight:

Latent feature: 4
(ID: 1296 , Weight: 2.292395125613071), (ID: 1204 , Weight: 2.220360388103401), (ID: 1724 , Weight:

Latent feature: 5
(ID: 600 , Weight: 2.3750805328994904), (ID: 814 , Weight: 2.370678296320661), (ID: 720 , Weight:
```

■

Latent Semantics & Image-Weight Pairs: Feature Descriptor: L3, Dimensionality Reduction: LDA, K=5

Task 6, Config 3: Layer3, LDA, K=5, Latent Semantics:

```
Top K latent semantics:
[[0.07930998 0.24953079 0.15898125 0.44782158 0.0643564 ]
 [0.07053774 0.26398878 0.15756403 0.46358711 0.04432234]
 [0.07269821 0.2559124 0.16136099 0.45744409 0.05258431]
 ...
 [0.23125428 0.1015543 0.13173872 0.09320021 0.44225249]
 [0.22109754 0.15514538 0.17318164 0.15056931 0.30000613]
 [0.25714241 0.08212903 0.11475878 0.06126809 0.48470169]]
Shape: (4339, 5)
```

Task 6, Config 3: Layer3, LDA, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 7928 , Weight: 0.2847279271499229), (ID: 7624 , Weight: 0.28472622867889047), (ID: 7680 , Weight:

Latent feature: 2
(ID: 7648 , Weight: 0.3328076133851687), (ID: 3590 , Weight: 0.29372159326955166), (ID: 5522 , Weight:

Latent feature: 3
(ID: 7648 , Weight: 0.25040508197805045), (ID: 5652 , Weight: 0.24294391511690203), (ID: 7192 , Weight:

Latent feature: 4
(ID: 610 , Weight: 0.6785701960401225), (ID: 624 , Weight: 0.6498951715438037), (ID: 436 , Weight:

Latent feature: 5
(ID: 7530 , Weight: 0.9665067598386337), (ID: 7548 , Weight: 0.8866632297564193), (ID: 7562 , Weight:

■
```

Latent Semantics & Image-Weight Pairs: Feature Descriptor: L3, Dimensionality Reduction: KMeans, K=5

Task 6, Config 4: Layer3, K-Means, K=5, Latent Semantics:

Top K latent semantics:

```
[ [ 71.22641065  88.56243153  24.7934815  63.09038871 136.9993852 ]
  [ 76.42781104  93.56308794  26.25367253  57.79964406 129.42065309]
  [ 65.3211771   91.55597332  33.91425626  65.59165325 147.77074921]
  ...
  [225.87746508 194.71368873 185.5321584 174.46968486 76.76875161]
  [ 90.03116118 70.49957244 86.43383638 55.74121146 88.35201499]
  [120.64052646 84.25763249 110.79549468 82.81762953 63.04821223]]
Shape: (4339, 5)
```

Task 6, Config 4: Layer3, K-Means, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 7530 , Weight: 480.3329394671072), (ID: 7548 , Weight: 412.512360588474), (ID: 7524 , Weight:

Latent feature: 2
(ID: 7530 , Weight: 450.97271856645256), (ID: 7548 , Weight: 381.39842417852304), (ID: 7524 , Weight:

Latent feature: 3
(ID: 7530 , Weight: 431.2213753820601), (ID: 7548 , Weight: 365.9558542779625), (ID: 7524 , Weight:

Latent feature: 4
(ID: 7530 , Weight: 426.37194062083086), (ID: 7548 , Weight: 359.2006798014107), (ID: 7524 , Weight:

Latent feature: 5
(ID: 7530 , Weight: 329.39721143636456), (ID: 7548 , Weight: 260.95026842565994), (ID: 7524 , Weight:

■
```


Latent Semantics & Image-Weight Pairs: Feature Descriptor: ResNet Output, Dimensionality Reduction: SVD, K=5

Task 6, Config 5: RESNET, SVD, K=5, Latent Semantics:

Top K latent semantics:

```
[[ 7.13702068  9.0789789  0.03725192 -3.56356979  0.01242555]
 [ 3.49423577  5.45995749  0.46352612 -3.76206563  1.06292192]
 [ 7.50995963 10.41956002  0.12842626 -4.9083201  -0.01767826]
 ...
 [ 6.35015837  4.32907137  0.28925199  1.19477057 -1.8380521 ]
 [ 7.51823265  3.82428052 -0.13733931  4.05000774 -2.45602846]
 [ 4.52990151  2.88578033 -0.53877476  1.89144267 -0.77640518]]
Shape: (4339, 5)
```

Task 6, Config 5: RESNET, SVD, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 2496 , Weight: 0.03153695371516463), (ID: 2626 , Weight: 0.03117960949001645), (ID: 2696 , Weight:

Latent feature: 2
(ID: 812 , Weight: 0.04150318138613264), (ID: 562 , Weight: 0.04128774150261516), (ID: 32 , Weight: 0.04128774150261516),

Latent feature: 3
(ID: 1838 , Weight: 0.04521456488344075), (ID: 1320 , Weight: 0.045009641863528016), (ID: 1588 , Weight: 0.045009641863528016),

Latent feature: 4
(ID: 6418 , Weight: 0.03745426615164149), (ID: 3120 , Weight: 0.034823368923093824), (ID: 3874 , Weight: 0.034823368923093824),

Latent feature: 5
(ID: 3234 , Weight: 0.04469728503952883), (ID: 7724 , Weight: 0.04374008759217053), (ID: 8500 , Weight: 0.04374008759217053),

==
```

Latent Semantics & Image-Weight Pairs: Feature Descriptor: ResNet Output, Dimensionality Reduction: NNMF, K=5

Task 6, Config 6: RESNET, NNMF, K=5, Latent Semantics:

```
Top K latent semantics:
[[2.69786107e-37 4.96392931e-02 1.00329221e-09 8.17894615e-01
  5.01233970e+00]
 [2.58210737e-01 6.05260011e-88 4.09996612e-02 4.46717967e-48
  3.27597203e+00]
 [6.82132068e-22 3.33565389e-02 1.83432197e-15 3.96859338e-01
  5.90024544e+00]
 ...
 [5.43149195e-24 5.48512468e-02 3.16626990e-01 2.34793448e+00
  1.96836481e+00]
 [7.87906921e-14 3.88127410e-02 1.95877535e-01 3.84987785e+00
  1.15256912e+00]
 [3.20984758e-27 1.17656111e-01 1.20802477e-24 2.13077998e+00
  1.03618018e+00]]
Shape: (4339, 5)
```

Task 6, Config 6: RESNET, NNMF, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 5756 , Weight: 5.565871947308715), (ID: 5740 , Weight: 5.451598668580194), (ID: 3028 , Weight:
Latent feature: 2
(ID: 2612 , Weight: 6.95842514889613), (ID: 2458 , Weight: 6.875731592535122), (ID: 2552 , Weight:
Latent feature: 3
(ID: 1586 , Weight: 6.887441246545942), (ID: 1758 , Weight: 6.865904491293591), (ID: 1320 , Weight:
Latent feature: 4
(ID: 2724 , Weight: 4.981847031506242), (ID: 5188 , Weight: 4.939444173920567), (ID: 3874 , Weight:
Latent feature: 5
(ID: 456 , Weight: 6.751895826441841), (ID: 32 , Weight: 6.730594363785725), (ID: 562 , Weight: 6.7088091
==
```

Latent Semantics & Image-Weight Pairs: Feature Descriptor: ResNet Output, Dimensionality Reduction: LDA, K=5

Task 6, Config 7: RESNET, LDA, K=5, Latent Semantics:

```
Top K latent semantics:
[[4.62668659e-04 9.93827438e-01 4.77093795e-03 4.66386642e-04
 4.72568269e-04]
 [4.35465993e-02 9.10718623e-01 2.18172415e-02 2.30555229e-02
 8.62012925e-04]
 [4.44416177e-04 9.98212273e-01 4.47304482e-04 4.47945140e-04
 4.48060874e-04]
 ...
 [5.34716727e-02 8.39549329e-01 5.45523900e-04 7.54529589e-02
 3.09805158e-02]
 [2.39339481e-02 5.79182374e-01 3.63362043e-02 3.43666760e-01
 1.68807137e-02]
 [6.80261337e-04 6.66788405e-01 1.17012887e-01 1.83491604e-01
 3.20268430e-02]]
Shape: (4339, 5)
```

Task 6, Config 7: RESNET, LDA, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 1594 , Weight: 0.998220645319878), (ID: 1354 , Weight: 0.9981706902718646), (ID: 1634 , Weight: 0.9981706902718646), (ID: 1354 , Weight: 0.9981706902718646), (ID: 1634 , Weight: 0.9981706902718646)

Latent feature: 2
(ID: 508 , Weight: 0.9985128767176167), (ID: 370 , Weight: 0.9985004774061779), (ID: 10 , Weight: 0.9985004774061779), (ID: 370 , Weight: 0.9985004774061779), (ID: 10 , Weight: 0.9985004774061779)

Latent feature: 3
(ID: 6142 , Weight: 0.9971283087991503), (ID: 3234 , Weight: 0.9969548833651968), (ID: 4224 , Weight: 0.9969548833651968), (ID: 3234 , Weight: 0.9969548833651968), (ID: 4224 , Weight: 0.9969548833651968)

Latent feature: 4
(ID: 6438 , Weight: 0.9963910484430735), (ID: 3828 , Weight: 0.9927858180183324), (ID: 3818 , Weight: 0.9927858180183324), (ID: 3828 , Weight: 0.9927858180183324), (ID: 3818 , Weight: 0.9927858180183324)

Latent feature: 5
(ID: 2258 , Weight: 0.9982034270899734), (ID: 2338 , Weight: 0.9981869444908399), (ID: 2274 , Weight: 0.9981869444908399), (ID: 2338 , Weight: 0.9981869444908399), (ID: 2274 , Weight: 0.9981869444908399)
```

Latent Semantics & Image-Weight Pairs: Feature Descriptor: ResNet Output, Dimensionality Reduction: KMeans, K=5

Task 6, Config 8: RESNET, K-Means, K=5, Latent Semantics:

Top K latent semantics:

```
[[101.70203328  77.15017289 105.63092207  66.66234574  31.32827793]
 [ 88.67532367  62.81872394  98.33940467  78.0945605  49.88766446]
 [106.33545522  83.84898371 111.03716183  74.58566111  23.61169492]
 ...
 [ 88.0119057  65.99304676  93.95369239  43.04651443  66.40308884]
 [101.86167659  80.05494487 103.79654902  44.98128332  88.49123727]
 [ 88.28642043  57.20897881  90.25979815  47.15191953  74.79762437]]
```

Shape: (4339, 5)

Task 6, Config 8: RESNET, K-Means, K=5, Image - Weight pairs:

Image - weight pairs sorted in descending order of weights for each latent feature:

```
Latent feature: 1
(ID: 3874 , Weight: 118.29992752454964), (ID: 6668 , Weight: 118.29690828059938), (ID: 6258 , Weight:

Latent feature: 2
(ID: 6668 , Weight: 100.38096216946668), (ID: 3466 , Weight: 97.48052538187109), (ID: 7324 , Weight:

Latent feature: 3
(ID: 6668 , Weight: 121.15351877131752), (ID: 812 , Weight: 120.3749656306906), (ID: 562 , Weight:

Latent feature: 4
(ID: 5018 , Weight: 100.56046082372941), (ID: 1350 , Weight: 99.8830495909902), (ID: 1244 , Weight:

Latent feature: 5
(ID: 2618 , Weight: 118.79762127699183), (ID: 2474 , Weight: 117.53481946348946), (ID: 2598 , Weight:
```

■

5.8 Task 7

5.8.1 Goal Description:

Implement a program which, given (a) an (even or odd numbered) imageID or an image file name, (b) a user selected latent semantics, and (c) positive integer k, identifies and visualizes the most similar k images, along with their scores, under the selected latent space.

5.8.2 Assumptions:

1. Any latent space can be used to find similar images, including label weight semantics.
2. The required latent semantics are already generated before executing this task.

5.8.3 Implementation Approach:

1. Given a query image, convert it to the feature space used in the generation of the latent space.
2. For LS1 and LS2:
 - (a) Reduce the query vector of this image using the latent space reducer.
 - (b) Find the K smallest distance vectors in the latent space, and display.
3. For LS3:
 - (a) First need to compute the image-label similarity vector using the original feature space.
 - (b) We can then reduce this vector to the latent semantics.
 - (c) We can then find the most similar label by finding the smallest distance vector in the latent space.
 - (d) We then find K most relevant images for that label using the representative label vectors for that latent space.
4. For LS4:

- (a) Generate the image image similarity vector for the input vector, we do this using the same we computed the image image similarity vector for every image in task 6.
- (b) Reduce the image image similarity vector using the reducer.
- (c) Find the K smallest distance vectors in the latent space, and display.

5.8.4 Interface Specifications:

1. To run this program, run all the cells in the task_7.ipynb file.
2. The program will first prompt to provide an image. You can either provide an image ID, or the name of an image in the inputs folder.
3. After this the program will prompt for the latent semantic technique with K as well as the feature space as an input for eg: if a latent semantic was generated in task 3 from color space with K as 5 and svd reduction the input would be "LS1_color_5_svd"
4. It will also provide a prompt for a valid K value representing the number of similar images that would be found for the input image.

5.8.5 Results:

5.9 Task 8

5.9.1 Goal Description:

Implement a program which, given (a) an (even or odd numbered) imageID or an image file name, (b) a user selected latent semantics, and (c) positive integer k, identifies and lists k most likely matching labels, along with their scores, under the selected latent space.

5.9.2 Assumptions:

1. Any latent space can be used to find similar images, including label weight semantics.
2. The required latent semantics are already generated before executing this task.

5.9.3 Implementation Approach:

1. Given a query image, convert it to the feature space used in the generation of the latent space.
2. For LS1 and LS2:
 - (a) Get the representative vectors for each label under the latent space.
 - (b) Find the K smallest distance vectors having unique labels in the latent space from the query's latent vector. These are our K labels.
3. For LS3:
 - (a) To utilize label-weight semantics, we first need to compute the image-label similarity vector using the original feature space.
 - (b) We do this by using the representative label vectors under the feature space.
 - (c) We can then construct a label similarity vector by finding the distance to each representative label vector (we take the closest).
 - (d) We can then reduce this vector to the latent semantics.
 - (e) Find the K smallest distance vectors having unique labels in the latent space from the query's latent vector. These are our K labels.

4. For LS4:
 - (a) We compute the image similarity vector for the query image. We do this by computing the distance from the query vector to each vector in the feature space.
 - (b) We then reduce this vector down to the latent space.
 - (c) Find the K smallest distance vectors having unique labels in the latent space from the query's latent vector. These are our K labels.

5.9.4 Interface Specifications:

1. To run this program, run all the cells in the task_8.ipynb file.
2. The program will first prompt to provide an image. You can either provide an image ID, or the name of an image in the inputs folder.
3. After this the program will prompt for the latent semantic technique with K as well as the feature space as an input for eg: if a latent semantic was generated in task 3 from color space with K as 5 and svd reduction the input would be "LS1_color_5_svd"
4. It will also provide a prompt for a valid K value representing the number of similar labels that would be found for the input image.

5.9.5 Results:

This task in particular performs poorly when the number of latent features is low. When we increased the latent feature size to 100 or more we got really good results. Additionally, we also get really good results in case of the resnet feature space. This could be because this feature space captures more non linearity in the data and hence gives a richer discriminating power.

Latent Semantics 4, Feature Descriptor: ResNet Avg-Pooling
Layer, Dimensionality Reduction: NNMf, K=5, Image ID = 0

LS3_avgpool_5_nnmf, k=10, image_id=0

```
K = 10  most similar labels by  LS3_avgpool_5_nnmf  semantics using distance:  cosine
1.      Label:  laptop                Distance:  0.00629513040423102
2.      Label:  cellphone             Distance:  0.023341425339257316
3.      Label:  camera                 Distance:  0.02385625623238774
4.      Label:  accordion              Distance:  0.03338076723751793
5.      Label:  stapler                Distance:  0.04000011165334372
6.      Label:  binocular              Distance:  0.07521303305324878
7.      Label:  revolver               Distance:  0.10430663740219226
8.      Label:  euphonium              Distance:  0.11356555682284641
9.      Label:  headphone              Distance:  0.11731184954171225
10.     Label:  watch                  Distance:  0.12534066746849004
```

Latent Semantics 4, Feature Descriptor: ResNet Avg-Pooling
Layer, Dimensionality Reduction: LDA, K=5, Image ID = 8676

LS3_avgpool_5_nnmf, k=10, image_id=8676

```
K = 10  most similar labels by  LS3_avgpool_5_nnmf  semantics using distance:  cosine
1.      Label:  cellphone             Distance:  0.00948784521371715
2.      Label:  laptop                Distance:  0.04750222288633443
3.      Label:  camera                 Distance:  0.04891932640236252
4.      Label:  accordion              Distance:  0.06481296823887694
5.      Label:  stapler                Distance:  0.06879430145233223
6.      Label:  headphone              Distance:  0.09397345614917507
7.      Label:  watch                  Distance:  0.10949649425458996
8.      Label:  binocular              Distance:  0.12287715041549319
9.      Label:  wrench                 Distance:  0.13223475791952422
10.     Label:  yin_yang               Distance:  0.13514985918445666
```

Latent Semantics 4, Feature Descriptor: L3, Dimensionality Reduction: NNMF, K=5, Image ID = 0

```
K = 10  most similar labels by LS4_layer3_5_lda semantics using distance: cosine
1.      Label:  Faces                      Distance:  0.00010615947739323062
2.      Label:  Faces_easy                  Distance:  0.0004285162034822765
3.      Label:  barrel                      Distance:  0.005421705635354823
4.      Label:  bonsai                      Distance:  0.006112784996985865
5.      Label:  pigeon                      Distance:  0.009721638121608644
6.      Label:  llama                       Distance:  0.010510338680026043
7.      Label:  hedgehog                    Distance:  0.0135083422359904
8.      Label:  ferry                       Distance:  0.015128948470517045
9.      Label:  sunflower                    Distance:  0.01577243009609519
10.     Label:  Leopards                    Distance:  0.01657122785284948
```

Latent Semantics 4, Feature Descriptor: L3, Dimensionality Reduction: LDA, K=5, Image ID = 8676

LS4_layer3_5_lda, k=10, image_id=8676

```
K = 10  most similar labels by LS4_layer3_5_lda semantics using distance: cosine
1.      Label:  garfield                    Distance:  3.238051935472441e-05
2.      Label:  watch                       Distance:  5.129007271797992e-05
3.      Label:  chair                       Distance:  5.8338179460548645e-05
4.      Label:  pigeon                      Distance:  6.229569835103366e-05
5.      Label:  crocodile                    Distance:  7.087642382008763e-05
6.      Label:  cougar_face                  Distance:  7.571907218684171e-05
7.      Label:  platypus                     Distance:  9.124209929023408e-05
8.      Label:  cannon                       Distance:  0.0001414730277929932
9.      Label:  umbrella                     Distance:  0.00015874038932628398
10.     Label:  butterfly                    Distance:  0.00018825499536245793
```

5.10 Task 9

5.10.1 Goal Description:

Implement a program which, given (a) a label l , (b) a user selected latent semantics, and (c) positive integer k , identifies and lists k most likely matching labels, along with their scores, under the selected latent space.

5.10.2 Assumptions:

1. Taking a mean of the latent space for each label would give the representative vector of that label in the latent space
2. This task depends on the input latent semantics to be generated successfully.

5.10.3 Implementation Approach:

1. For LS1, we have an image x latent space vector. We find representative vector of each label in the latent space by taking an average of the image latent vectors belonging to the label. After that we find the distances between each representative vector from the input label representative vector and sort it in an ascending order. This gives us the top K closest labels to a particular given label.
2. For LS2, on running CP decomposition we get an image x latent space vector, a label x latent space vector and a feature x latent space vector along with a factor matrix. For the purpose of finding the closest representative vector to a given label we just find the distance between the vector corresponding to the input label and all other vectors in the label x latent space vector. Then we just sort the distances in an ascending order and output the top K distances.
3. In a similar way, for LS3, we have an label x latent space vector. Hence we find distances between the vector corresponding to the label and the other labels and then sort it in an ascending order.
4. For LS4, we have an image x latent space vector, here we find the representative vector for each label by taking an average of the image latent vectors belonging to a that label. After that we just find the

distances between the input label representative vector and the other representative vectors and sort it in an ascending order.

5.10.4 Interface Specifications:

1. To run this program, run all the cells in the task_9.ipynb file.
2. The program will first prompt to provide a valid label (in the same format of the Caltech101 dataset) or an integer representing a label.
3. After this the program will prompt for the latent semantic technique with K as well as the feature space as an input for eg: if a latent semantic was generated in task 3 from color space with K as 5 and svd reduction the input would be "LS1_color_5_svd"
4. It will also provide a prompt for a valid K value representing the number of similar labels that would be found for the input label.

5.10.5 Results:

For LS1, in the color feature space we selected the cityblock distance function and for the resnet feature space we selected the cosine distance function.

For LS2, in the HoG as well as resnet feature space we selected the cosine distance function.

For LS3, we selected cosine distance for the avgpool and resnet feature spaces. For LS4, we selected the cosine distance for the layer3 and resnet feature spaces.

We primarily selected these distance functions through testing with multiple labels and inputs, this took a long time but it ensured that we got good accuracy for our test labels. For the rest we defaulted to the distance functions selected for the particular feature space.

In case of the input label of airplanes we get a result of airplanes, helicopter, ferry, ketch and schooner as closest labels. The relation between airplanes and helicopters is fairly straightforward unlike the other labels. However, in case of ferry, ketch and schooner's the background surrounding the images is mostly blue which would explain the outputs since the representative feature of the airplanes label would probably have sky blue as its background. We also experimented with other labels and found similarly good results.

Latent Semantics 1, Feature Descriptor: ResNet Output, Dimensionality Reduction: Kmeans (k=5), K=10, Label ID = 0

LS1_resnet_5_kmeans, k=10, label = 0

```
K = 10 most similar labels by LS1_resnet_5_kmeans latent semantics using distance: cosine
1.      Label: Faces      Distance: 0
2.      Label: Faces_easy Distance: 0.000657288055061378
3.      Label: yin_yang   Distance: 0.017300378174211373
4.      Label: pizza      Distance: 0.018479955385930613
5.      Label: panda      Distance: 0.01849288337451649
6.      Label: garfield   Distance: 0.018656414298127033
7.      Label: snoopy     Distance: 0.018668950251181315
8.      Label: cellphone  Distance: 0.018695814085765483
9.      Label: pigeon     Distance: 0.019778714057928415
10.     Label: stapler    Distance: 0.01985385507274917
```

Latent Semantics 1, Feature Descriptor: ResNet Output, Dimensionality Reduction: Kmeans (k=5), K=10, Label ID = 20

LS1_resnet_5_kmeans, k=10, label = 20

```
K = 10 most similar labels by LS1_resnet_5_kmeans latent semantics using distance: cosine
1.      Label: beaver     Distance: 0
2.      Label: wild_cat   Distance: 1.830917280631894e-05
3.      Label: brontosaurus Distance: 0.00010988608283069023
4.      Label: dolphin    Distance: 0.00012809186763518277
5.      Label: pigeon     Distance: 0.00014607009693834438
6.      Label: crocodile  Distance: 0.00017346163800346925
7.      Label: dalmatian  Distance: 0.00021224326636637425
8.      Label: cup        Distance: 0.00022896406784289614
9.      Label: rooster    Distance: 0.00026547012484723354
10.     Label: platypus   Distance: 0.00027261899958230806
```

Latent Semantics 1, Feature Descriptor: ResNet Output, Dimensionality Reduction: Kmeans (k=5), K=10, Label ID = 100

LS1_resnet_5_kmeans, k=10, label = 100

```
K = 10 most similar labels by LS1_resnet_5_kmeans latent semantics using distance: cosine
1.      Label: yin_yang      Distance: 0
2.      Label: snoopy      Distance: 3.3391275384264674e-05
3.      Label: garfield      Distance: 0.00011951192018688772
4.      Label: dollar_bill    Distance: 0.00025879806960782403
5.      Label: scissors      Distance: 0.00026000176945217834
6.      Label: cellphone      Distance: 0.000342910249433781
7.      Label: stapler      Distance: 0.00046018947080017636
8.      Label: wrench      Distance: 0.0006394054359869505
9.      Label: headphone      Distance: 0.0007977145575985833
10.     Label: minaret      Distance: 0.0008151468970692965
```

Latent Semantics 3, Feature Descriptor: ResNet Output, Dimensionality Reduction: Kmeans (k=5), K=10, Label ID = 0

LS3_resnet_5_kmeans, k=10, label = 0

```
K = 10 most similar labels by LS3_resnet_5_kmeans latent semantics using distance: cosine
1.      Label: Faces      Distance: 0
2.      Label: Faces_easy    Distance: 0.00013575609883664796
3.      Label: soccer_ball    Distance: 0.0002096563868142498
4.      Label: euphonium      Distance: 0.0009221555252149072
5.      Label: saxophone      Distance: 0.0011558723112041758
6.      Label: watch      Distance: 0.0016089721936273227
7.      Label: laptop      Distance: 0.0019271661836816056
8.      Label: chair      Distance: 0.0024772260109335864
9.      Label: accordion      Distance: 0.0025283979306390814
10.     Label: umbrella      Distance: 0.002887273941816426
```

Latent Semantics 3, Feature Descriptor: ResNet Output, Dimensionality Reduction: Kmeans (k=5), K=10, Label ID = 20

LS3_resnet_5_kmeans, k=10, label = 20

```
K = 10 most similar labels by LS3_resnet_5_kmeans latent semantics using distance: cosine
1.      Label: ceiling_fan      Distance: 0
2.      Label: yin_yang         Distance: 0.00023649631131850146
3.      Label: gramophone       Distance: 0.0003880147195332029
4.      Label: dollar_bill      Distance: 0.00041926222514443445
5.      Label: scissors        Distance: 0.0012753603494212173
6.      Label: metronome        Distance: 0.0012781900588283435
7.      Label: mandolin         Distance: 0.0014414405358855742
8.      Label: snoopy           Distance: 0.001796495773341844
9.      Label: wrench           Distance: 0.003199129285612834
10.     Label: trilobite        Distance: 0.003298032280808938
```

Latent Semantics 3, Feature Descriptor: ResNet Output, Dimensionality Reduction: Kmeans (k=5), K=10, Label ID = 100

LS3_resnet_5_kmeans, k=10, label = 100

```
K = 10 most similar labels by LS3_resnet_5_kmeans latent semantics using distance: cosine
1.      Label: yin_yang         Distance: 0
2.      Label: dollar_bill      Distance: 0.0002329849308065901
3.      Label: ceiling_fan      Distance: 0.00023649631131850146
4.      Label: gramophone       Distance: 0.0005797652794052865
5.      Label: scissors        Distance: 0.00111899777274356222
6.      Label: snoopy           Distance: 0.0016568420197341505
7.      Label: mandolin         Distance: 0.0017187561595993772
8.      Label: metronome        Distance: 0.0018163860891786676
9.      Label: wrench           Distance: 0.003495097090496113
10.     Label: menorah          Distance: 0.003562250430394598
```

5.11 Task 10

5.11.1 Goal Description:

Implement a program which, given (a) a label l , (b) a user selected latent semantics, and (c) positive integer k , identifies and lists k most relevant

images, along with their scores, under the selected latent space.

5.11.2 Assumptions:

1. Taking a mean of the latent space for each label would give the representative vector of that label in the latent space
2. In case of an input with LS3, the code would take about 5 minutes to run since the image x label matrix would have to be generated which would be around 4339 (number of images) x 101 (number of labels) = 438,239 iterations
3. This task depends on the input latent semantics to be generated successfully.

5.11.3 Implementation Approach:

1. For LS1, LS2 and LS4 we had the input in the form of images x latent space. Hence, we found representative image for the input label by taking a mean of the images belonging to that label. After that we just found the distance between each image and the representative image for the input label and sorted it in ascending order.
2. For LS3, we had the input in the form of label x latent space. To get images in the form of image x latent space we first found the image x label similarity matrix. After this we ran the same feature reduction technique to convert the image x label similarity matrix into a image x latent space matrix. Lastly, we found the distances between the label corresponding to the input label (in the label x latent space matrix) and each image in the image x latent space matrix. We then sorted it in ascending order and displayed the results.

5.11.4 Interface Specifications:

1. To run this program, run all the cells in the task_10.ipynb file.
2. The program will first prompt to provide a valid label (in the same format of the Caltech101 dataset) or an integer representing a label.

3. After this the program will prompt for the latent semantic technique with K as well as the feature space as an input for eg: if a latent semantic was generated in task 3 from color space with K as 5 and svd reduction the input would be "LS1_color_5_svd"
4. It will also provide a prompt for a valid K value representing the number of similar labels that would be found for the input label.

5.11.5 Results:

For LS1, in the resnet feature space we selected the euclidean distance function.

For LS2, in the resnet feature space we selected the cityblock distance function.

For LS3, we selected cosine distance for the resnet feature spaces.

For LS4, we selected the cosine distance for the layer3 and resnet feature spaces.

We primarily selected these distance functions through testing with multiple labels and inputs, this took a long time but it ensured that we got good accuracy for our test labels. For the rest we defaulted to the distance functions selected for the particular feature space.

As seen in the output with label airplanes, and feature space resnet we get airplanes as the output. However, it is interesting to note that not all airplanes close to the representative image have sky blue backgrounds. This could be because of outliers having different color backgrounds pulling the mean away from the sky blue background. We experimented with other labels and feature spaces and found good results. However, in case of LS2 we did get some other label images in our outputs for some labels. This could be because of the low reconstruction accuracy in CP decomposition.

5.12 Task 11

5.12.1 Goal Description:

Task 11 Implement a program which (a) given a feature model or latent space, (b) a value n , (c) a value m , and (d) a label l ,

1. creates a similarity graph, $G(V, E)$, where V corresponds to the images in the database and E contains node pairs v_i, v_j such that, for each subject v_i , v_j is one of the n most similar images in the database in the given space,
2. identifies the most significant m images (relative to the given label l) using personalized PageRank measure. Use [8] as reference.

5.12.2 Assumptions:

1. The code assumes that the distance matrix has been pre-computed and cached. If it doesn't exist, the code computes it, which may take 3-10 minutes. This assumption is made to optimize execution time by avoiding re-computation whenever possible.
2. For the feature spaces, the distance matrix is either computed or retrieved from cache. The cache functionality was incorporated since distance computation between $4k \times 4k$ takes between 3-10 mins.
3. For the latent spaces LS1 and LS2, the appropriate dimensionality reducer (SVD, NNMF, etc) are deserialized and used to obtain the image-image distance matrix.
4. For latent space LS3, it is assumed that the label-to-label similarity matrix is already precomputed from Task 5 and cached. The pre-computed distance matrix is fetched, and then dimensionality reduction is applied.
5. For latent space LS4, it is assumed that the distance matrix for the original feature space is cached. The precomputed distance matrix is fetched and then dimensionality reduction is applied.

5.12.3 Implementation Approach:

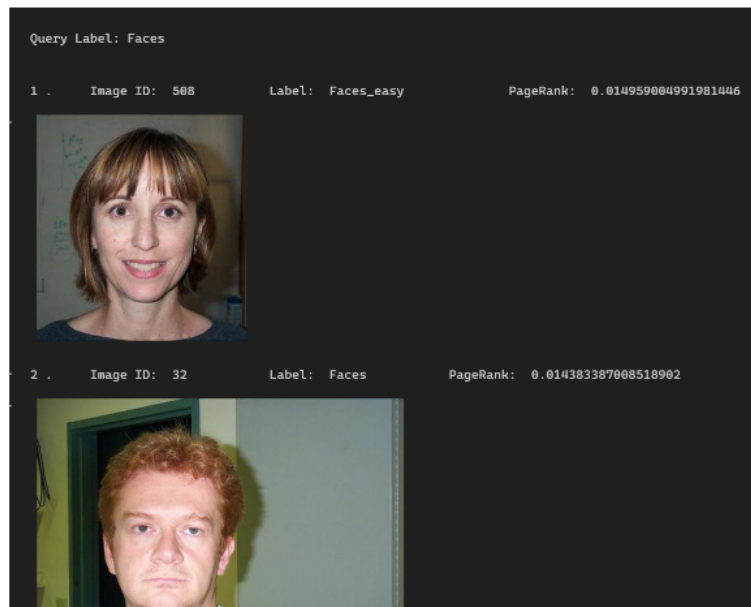
1. The program begins by taking user input for the feature/latent space, 'n' - the count of most similar images in the database in the given space, 'm' - the count of most significant images in the label to be returned as output of Personalized PageRank Algorithm and 'l' - the label for which we want to find similar labels.
2. The program then retrieves the feature vectors or latent space vectors based on the user's input. If the feature space is one of the predefined ones ('color', 'hog', 'avgpool', 'layer3', 'fc', 'resnet'), the feature vectors are directly obtained from the database.
3. If the feature space is latent ('LS1_color_5_svd' or similar), the program retrieves the reducer and image feature vectors, then computes the feature space.
4. The program checks if the distance matrix exists in cache and retrieves it or computes it, depending on whether it's cached.
5. A similarity graph is generated based on the 'n' most similar images in the feature space, and it's represented as an adjacency matrix.
6. Personalized PageRank is computed for the similarity graph with a personalized probability vector 's' associated with the provided 'label'.
7. The value for the damping factor was set to 0.85. It is a common choice in PageRank algorithms. It ensures that there is an 85% probability of transitioning to a linked page and a 15% probability of teleporting to any page in the graph. This balance facilitates both exploration and exploitation during ranking.
8. The program uses a fixed number of 50 iterations for the PageRank algorithm. While the number of iterations can vary, 50 iterations are sufficient for most practical purposes and provide a reasonable trade-off between accuracy and computation time.
9. The top 'm' significant images related to the label are determined based on the PageRank scores.
10. The program then displays the top 'm' images with their image IDs, labels, and PageRank scores.

5.12.4 Interface Specifications:

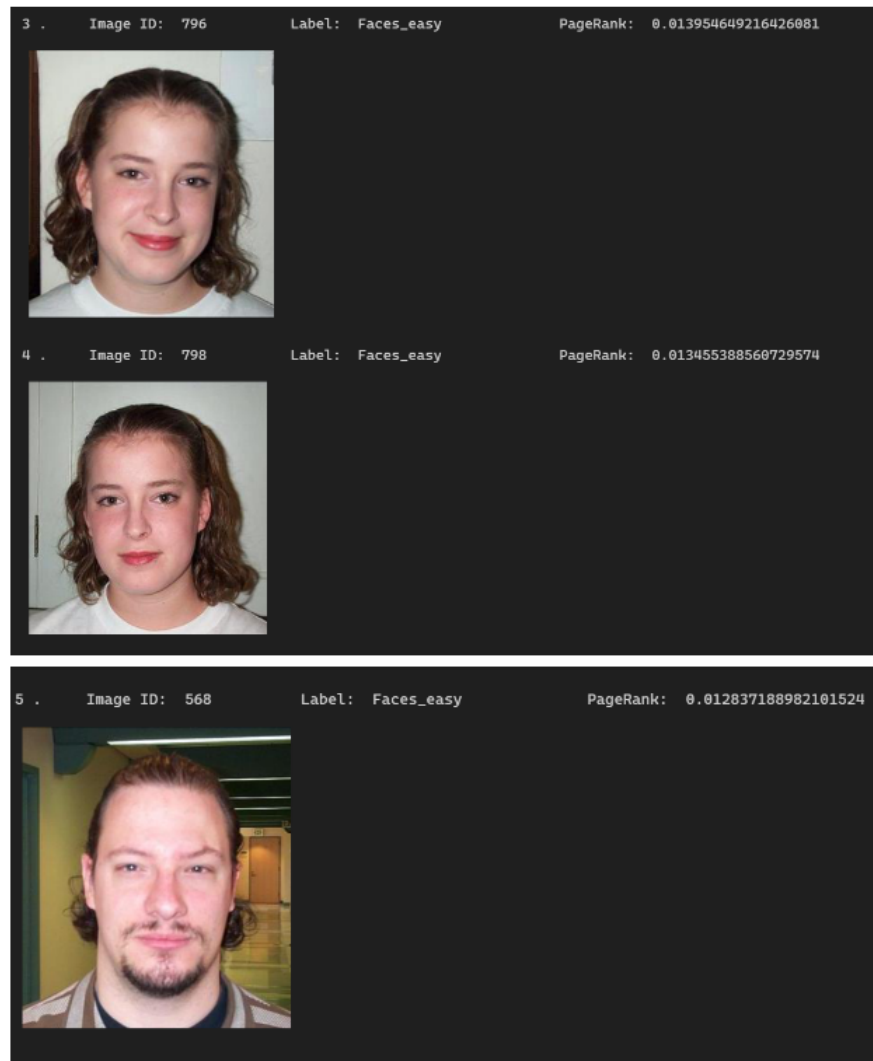
1. For the first input, you can either choose a stand-alone feature vector OR a combined latent semantic generated for a particular feature vector (with K value & dimensionality reduction technique).
2. Second, input the number of similar images you wish to use from the database.
3. Third, input the number of significant images in the label to be returned as output of the Personalized PageRank Algorithm.
4. Fourth, Enter the label ID for which you want to find similar labels.

5.12.5 Results:

Results for Label ID: 0 - Feature Descriptor: ResNet FC, $n = 10$, $m = 5$ - Ranks 1-2



Results for Label ID: 0 - Feature Descriptor: ResNet FC, $n = 10$, $m = 5$ - Ranks 3-5



Results for Label ID: 20 - Feature Descriptor: ResNet FC, n = 10, m = 5 - Ranks 1-3

Query Label: ceiling_fan

1 .	Image ID: 3694	Label: ceiling_fan	PageRank: 0.05415411273422672
			
2 .	Image ID: 3664	Label: ceiling_fan	PageRank: 0.048900292368648846
			
3 .	Image ID: 3678	Label: ceiling_fan	PageRank: 0.044685211021639686
			

Results for Label ID: 20 - Feature Descriptor: ResNet FC, $n = 10$, $m = 5$ - Ranks 4-5

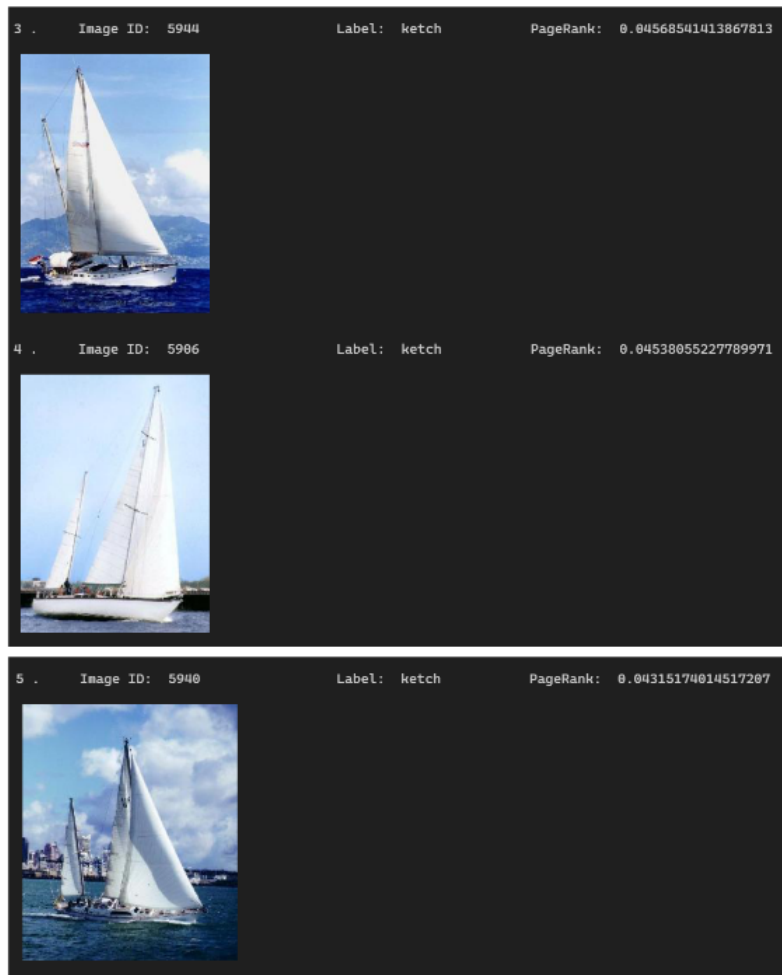
4 .	Image ID: 3686	Label: ceiling_fan	PageRank: 0.04339293089160453
			
5 .	Image ID: 3676	Label: ceiling_fan	PageRank: 0.03958414494396778
			

Results for Label ID: 55 - Feature Descriptor: ResNet FC, $n = 10$, $m = 5$ - Ranks 1-2

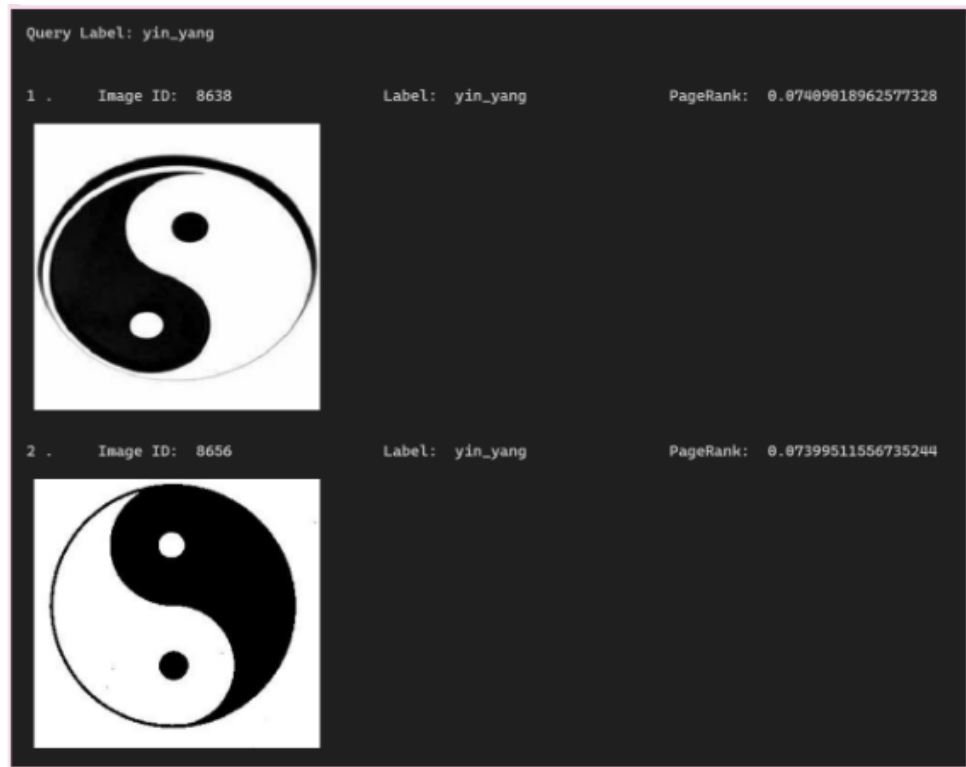
Query Label: ketch

1 .	Image ID: 5928	Label: ketch	PageRank: 0.051551116691007065
			
2 .	Image ID: 5986	Label: ketch	PageRank: 0.04615693720884917
			

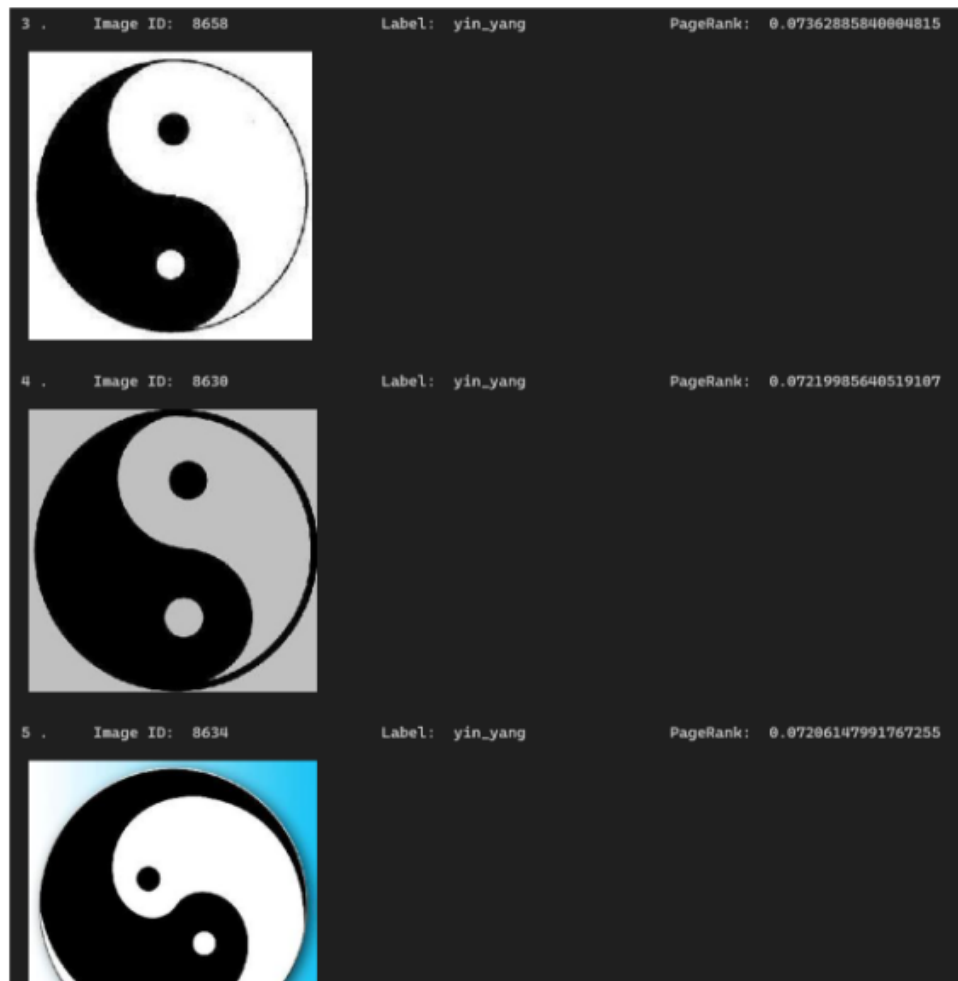
Results for Label ID: 55 - Feature Descriptor: ResNet FC, $n = 10$, $m = 5$ - Ranks 3-5



Results for Label ID: 100 - Feature Descriptor: ResNet FC, n
= 10, m = 5 - Ranks 1-2



Results for Label ID: 100 - Feature Descriptor: ResNet FC, $n = 10$, $m = 5$ - Ranks 3-5

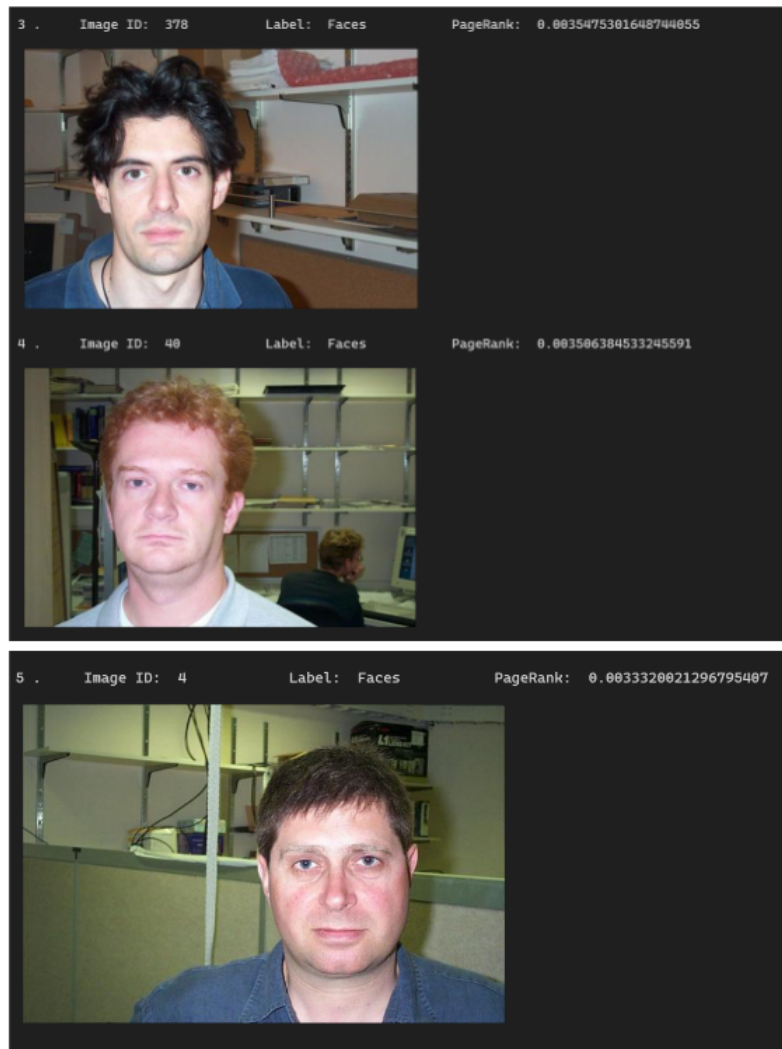


Results for Label ID: 0 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 1-2

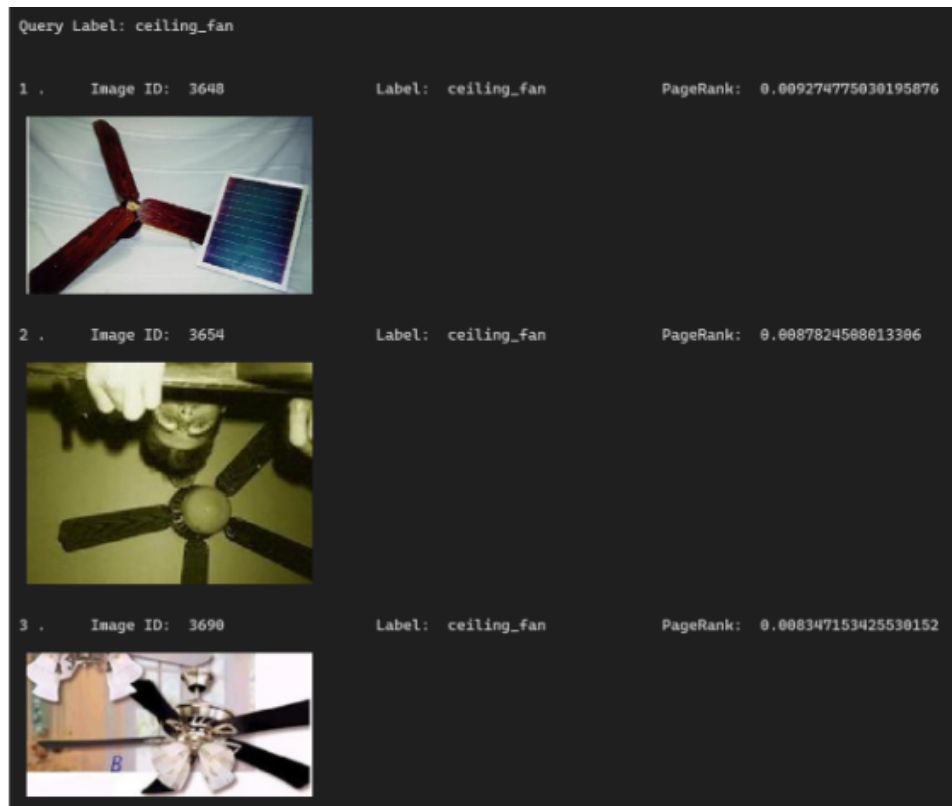
Query Label: Faces

1 .	Image ID: 406	Label: Faces	PageRank: 0.004379226074565073
			
2 .	Image ID: 204	Label: Faces	PageRank: 0.003985225827216408
			

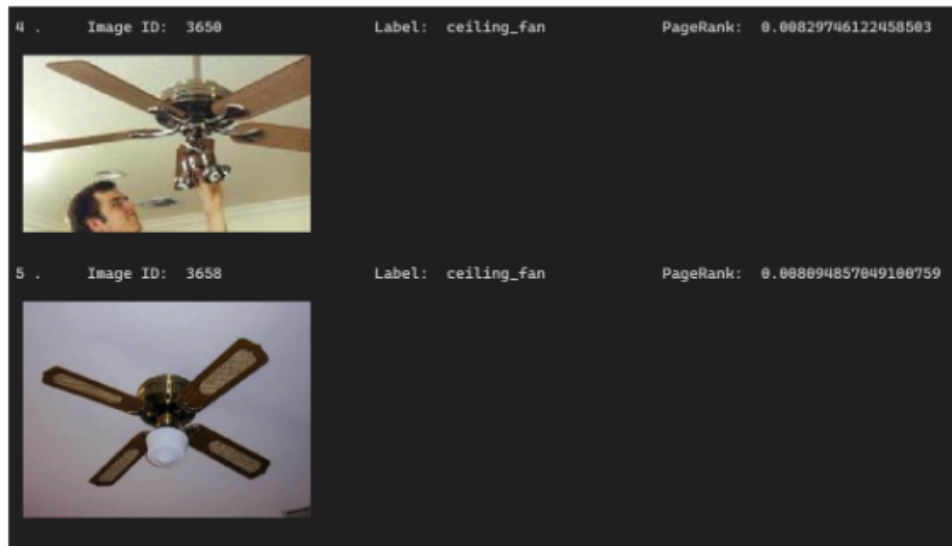
Results for Label ID: 0 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 3-5



Results for Label ID: 20 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 1-3



Results for Label ID: 20 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 4-5



Results for Label ID: 55 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 1-2

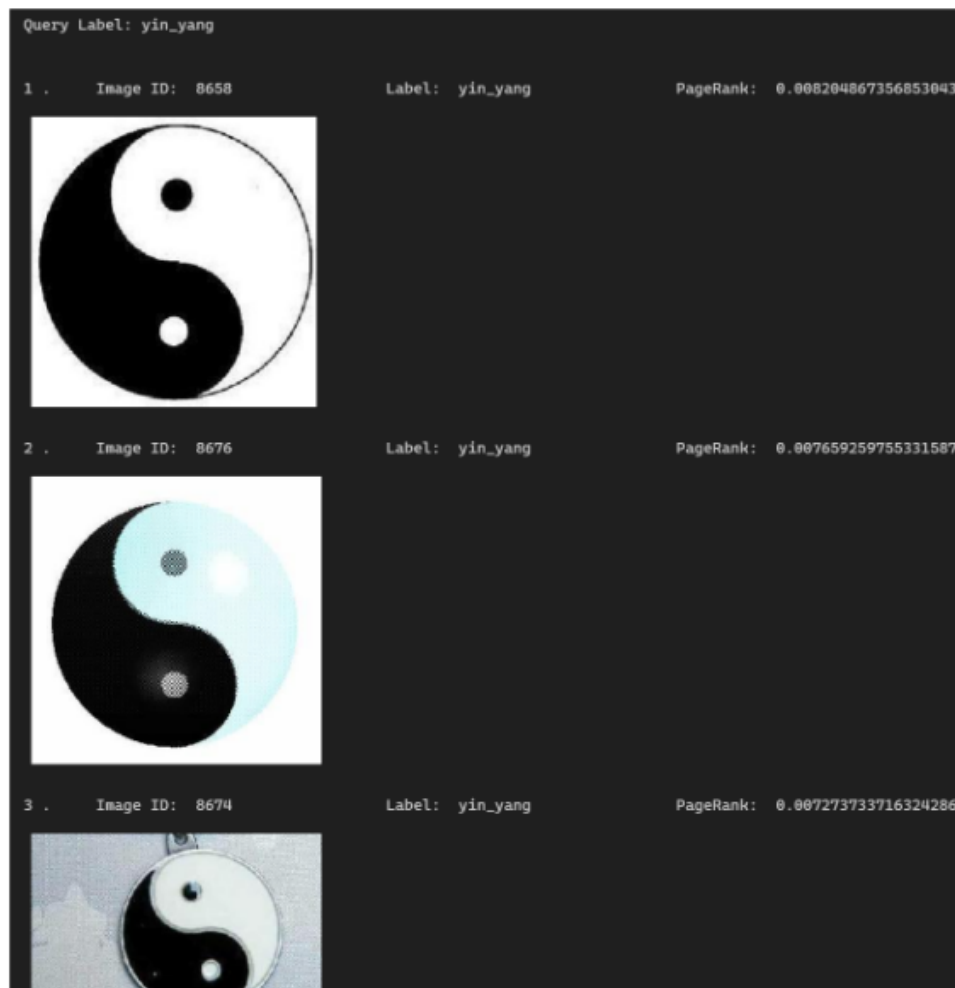
Query Label: ketch

1 .	Image ID: 5886	Label: ketch	PageRank: 0.011944998588902671
			
2 .	Image ID: 5898	Label: ketch	PageRank: 0.009314159538974427
			

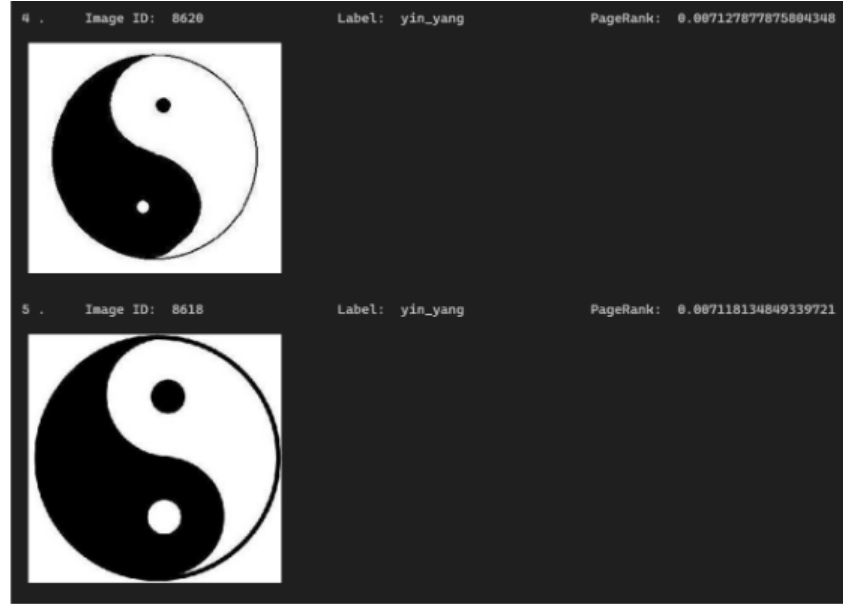
Results for Label ID: 55 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 3-5

3 .	Image ID: 5978	Label: ketch	PageRank: 0.008889810396507013
			
4 .	Image ID: 5908	Label: ketch	PageRank: 0.008277312436396511
			
5 .	Image ID: 5952	Label: ketch	PageRank: 0.008239401784662325
			

Results for Label ID: 100 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 1-3



Results for Label ID: 100 - Latent Semantic: LS1, Feature Descriptor: Color, Dimensionality Reduction: SVD, $K = 5$, $n = 10$, $m = 5$ - Ranks 4-5



5.12.6 Key Observations:

1. For all feature/latent spaces except LS3, the distance matrix is a 4339×4339 matrix where any entry V_{ij} is the distance between two images i and j in the given space.
2. It was observed that the distance matrix computation took a significant amount of time, and hence the decision was made to cache the intermediate distance matrix outputs.
3. Another intermediate output of the program is transition matrix. This was constructed from the distance matrix by assigning a value of 1 to the nearest N neighbors of the corresponding row and setting all other values to 0.
4. The 'pr' vector is personalized, meaning it considers the specified label 'l' during the PageRank computation. This personalization ensures

that the PageRank scores are relative to the provided label, making it especially useful for finding images that are significant in the context of that label.

5. The PageRank scores in 'pr' are normalized probabilities, and they sum up to 1. This means that if you sum all the PageRank scores in 'pr,' you will get 1, indicating a probability distribution.
6. The outputs for various combinations of feature space and label combinations provided very good results since PageRank is able to take an aggregate across all images within the label and provide the most representative image. Even in the case of label-label similarity, it computed labels that were meaningfully related to the input label.
7. Giving a small value of N generally tended to give more accurate results, since the random walk would be restricted to only the closest nodes and therefore images from other labels wouldn't be connected.
8. Some results were surprising, for example, a platypus image was occasionally a top result for the Face label for certain feature spaces. This could be because feature spaces like "color" and "hog" may not be optimized for facial recognition, and the features they capture might include color or texture characteristics that are shared between unrelated objects. Therefore, a platypus image, which might share some visual features with faces (e.g., color patterns), occasionally appear as a top result.

6 System Requirements & Execution Instructions

The System & Dependency Requirements, and Execution Instructions are provided in the README.md file provided with this report.

Furthermore, clear information on how to visualize the outputs for each respective task is also mentioned in the README.md file.

Justifications for choosing a particular visualization standard for each major task were provided earlier in this Report.

Code quality, clear documentation and reports, & an easy-to-use user experience were prioritised during the development of this entire phase of the

project.

We also hope that our code is easily extensible for future Phases of this project. Note: Instead of hope we should say goal During development of the project, our goal was to ensure our code was easily extensible for future phases of the project as the beginning part of this project incorporated some aspects of Phase 1.

Moreover, for System Requirements, (to be more specific), this code repository utilizes Jupyter Notebooks developed in Visual Studio Code, Python. The code has been tested on Python versions 3.9 - 3.11, due to using the latest packages, we recommend using the later versions of Python such as 3.9 or higher. Additionally, we ran our code in venv Python package, regular Python installation, and MiniConda Python environment and both latest builds of Mac OS and Windows 11.

Any reasonably modern hardware should be able to execute all the tasks, albeit slowly relative to the performance. We have found acceptable performance on Apple Silicon M2 Pro with 16GB unified ram. We have also verified on 10th generation Intel Core i7 10700k and 16GB ram. A GPU is not required. Sufficient storage of at least 30 GB is recommended to execute the project.

7 Related Works

In order to accomplish the objective of retrieving similar images, the authors [11] utilized the BigTransfer Model (commonly known as BigTransfer or BiT), an advanced deep learning model. BiT is essentially a modified version of the ResNet architecture, pre-trained on extensive datasets such as ImageNet and ImageNet-21k, endowing it with formidable feature extraction capabilities.

The proposed method involves harnessing the fine-tuned pre-trained Convolutional Neural Network (CNN) model to extract essential features from images. These extracted features are then utilized to train a K-Nearest Neighbor (K-NN) model. This K-NN model plays a crucial role in identifying images that are most akin to a given query image. The primary purpose of this model is to efficiently locate similar images, a task that can be challenging when using text-based queries. Notably, this process is accomplished within a remarkably short inference time.

The authors of [12] propose a method for computing personalized PageRanks efficiently in batch mode, particularly focusing on one-hop personalized PageRanks. One-hop personalized PageRanks refer to the importance scores of immediate neighbors of a given node, personalized to a specific user or context.

The approach presented in the paper optimizes the computation process, allowing for efficient batch processing of these personalized PageRanks. By doing so, the method enhances the speed and scalability of computing personalized PageRanks, making it applicable in large-scale networks and real-time applications.

8 Conclusions

1. In conclusion, we learned a lot from phase 2. For example, since each later task utilized previous tasks results, we learned how to effectively communicate, improved our skills with write modular code, and utilize our time by efficiently sharing tasks by analyzing common parts / functions within all the tasks. Moreover, we all improved our report writing skills to expertly communicate every single aspect of our research, implemenation, and results of our programs based on feedback provided from phase 1. Additionally, we gained a better understanding of dimensionality such as SVD, NNMF, LDA, k-means, and CP-decomposition and latent semantics through the implementation in the tasks that asked for those techniques.
2. Some tasks are very time consuming, especially those involving Image-Image similarity decomposition, and LDA.
3. Lower values of K during feature reduction produced worse results. For the sake of this report, we used the K values provided, but we observed that we got substantially better results when we chose K to be much higher (say half the original feature space size).
4. Whenever we had to use an incompatible latent semantic with a task, the accuracy was poor, since we first had to map the query to the latent semantic, which results in some loss. For example, in task 7, using label weight semantics proved to not provide good results, since we first had

to classify the query image to a label, and then use the label weight semantics to find relevant images for the label.

References

- [1] SciPy Spatial Distance (Distance Measures) Formulae, "<https://docs.scipy.org/doc/scipy/reference/spatial.distance.html>"
- [2] Education Ecosystem (Towards Data Science), K-Means Clustering in Machine Learning, "<https://towardsdatascience.com/understanding-k-means-clustering-in-machine-learning-6a6e67336aa1>"
- [3] Soner Yildirim (Towards Data Science), DBSCAN Clustering, "<https://towardsdatascience.com/dbscan-clustering-explained-97556a2ad556>"
- [4] Web MIT Lectures, Singular Value Decomposition, "https://web.mit.edu/be.400/www/SVD/Singular_Value_Decomposition.htm"
- [5] Scikit-Learn Decomposition Modules - NMF, "<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.NMF.html>"
- [6] Blei, David M., Ng, Andrew Y., Jordan, Michael I. (2003). Latent Dirichlet Allocation
- [7] Tensor (CP) Decomposition, https://tensorly.org/stable/user_guide/tensor_decomposition.htm
- [8] Huang, S., Li, X., Candan, K. S., Sapino, M. L. (2016). Reducing seed noise in personalized PageRank. Social Network Analysis and Mining, 6(1), 1-25.
- [9] Minster, Rachel, et al. "CP Decomposition for Tensors via Alternating Least Squares with QR Decomposition." Numerical Linear Algebra with Applications, 5 June 2023, <https://doi.org/10.1002/nla.2511>. Accessed 16 Oct. 2023.
- [10] Choi, Jonghyun, et al. Toward Sparse Coding on Cosine Distance.
- [11] Nath, S., Nayak, N. (2022, May 18). Identical image retrieval using Deep Learning. arXiv.org. <https://arxiv.org/abs/2205.04883>

- [12] Luo, Siqiang , et al. “Efficient Batch One-Hop Personalized PageRanks.” Ieeexplore.ieee.org, ieeexplore.ieee.org/document/8731411. Accessed 16 Oct. 2023.

A Specific Roles and Details of the Group Members

Everyone -

1. Everyone in the group contributed to discussions about tasks, implementations, general utility functions to be used in other tasks, and document. However, each group member took lead on similar tasks and did the bulk of the implementation for each one.

Caleb Panikulam - cpanikul@asu.edu -

1. Task 2b - Implemented user interface, extraction of ResNet Output for each image, and research (and tested) which implementation of k-means and k-nearest-neighbors would be best for this task.
2. Task 8 - Implemented user interface, Researched (and tested) different clustering algorithms such as DBScan and K-Means to see which one would be best for this task.
3. Task 10 - Researched (and tested) ideas for handling the label-weight matrix from LS3 in order to find similar labels.
4. Report - Took co-lead on writing report and took input from group members.

Gokul Vasudeva - gvasude2@asu.edu -

1. - Task 0a - Utilized Gokul's code from Phase 1 to store the imageID, feature vectors, and original image labels for the requests 5 different feature spaces.
2. Task 0b - Implemented user interface, found similar k images for a given image and displayed images in Jupyter Notebook with the scores.
3. - Task 3 - Implemented user interface, the 4 requested dimension reduction techniques, return the top-k latent semantics, save that latent semantics in a pickle file, and list the imageID-weight pairs in decreasing pairs.

4. - Task 7 - Implemented user interface, found the k similar images under the given latent space and displayed them with scores using Jupyter Notebook.
5. - Task 8 - Found the k similar labels under the given latent space and displayed them with scores using Jupyter Notebook. Also, utilized Caleb's research for Task 8 to identify the best approach to find the k similar labels.

Joshua Martin Noronha - jnoronha@asu.edu -

1. - Task 4 - Implemented user interface, the CP-decomposition of a 3-modal tensor, return the top-k latent semantics, save that latent semantics in a pickle file, and list the label-weight pairs in decreasing pairs, save that latent semantics in a pickle file, and list the label-weight pairs in decreasing pairs.
2. Task 5 - Implemented user interface, create and save label-label similarity matrix, performs user selected dimensionality reduction technique on this label-label matrix, save the latent semantics in a file, and list label-weight pairs in decreasing order of weights.
3. Task 9 - Implemented user interface, found the k similar labels under the given latent space using distance formulas and displayed them with scores using Jupyter Notebook.
4. Task 10 - Implemented user interface, found the k similar images under the given latent space using distance formulas and displayed them with scores using Jupyter Notebook.

Kiran Sthanusubramonian - ksthanus@asu.edu - 1225495115

1. - Helped organize code in each task and repository with goal of easy extensible for later tasks in Phase 2 and future phases.
2. Report - Took co-lead on writing report and took input from group members.

Sandipan De - sandipan@asu.edu -

1. Task 1 - Implemented user interface, found similar k images for a given label and displayed images in Jupyter Notebook with the scores.

2. Task 2a - Implemented user interface, and find similar labels based on user selected feature space, and research (and tested) which implementation of k-means for this task.
3. Task 6 - Implemented user interface, create and save image-image similarity matrix, performs user selected dimensionality reduction technique on this image-image matrix, save the latent semantics in a file, and list image-weight pairs in decreasing order of weights.

Shankar Harinarayanan - sharina1@asu.edu -

1. Task 11 - Since task 11 was disconnected from the rest of the tasks and Page Rank required a lot of research, Shankar volunteered to focus his attention only on Task 11 and made sure to fulfill all tasks required in the instructions with great accuracy.
2. Report - Took co-lead on collecting output for various tasks, analyzing them and added them to the report